

E-commerce system MVP design scheme and technical understanding

Name: Lawrence Zhou Date: June 2026

I. User Roles & Core Scenarios

1.1 Project Background and Product Objectives

This project aims to build a basic e-commerce system from scratch, enabling consumers to browse products, place orders, make payments, and track orders, while supporting operations staff in managing products, inventory, orders, and shipments.

According to the requirements of the question, the first phase of this system should focus on the basic transaction closed loop:

Browse Products → Add to Cart / Buy Now → Submit Order → Pay → Ship → Complete Order

Therefore, this design will not aim to build a complete large-scale e-commerce platform all at once, but will instead focus on MVP, prioritizing the verification of whether the system can reliably complete a real transaction.

From a product manager's perspective, the most important aspect of the MVP phase is not the number of features, but rather the usability of the core workflow, the clarity of the status, and the accuracy of the data. For e-commerce systems, if products can be displayed but orders cannot be placed, or if users have paid but the order status is out of sync, it will directly disrupt the transaction experience. Therefore, this solution will prioritize key modules such as products, SKUs, inventory, orders, payment, and shipping, while growth-oriented features such as coupons, membership systems, product reviews, and personalized recommendations will be placed in subsequent iterations.

The product objectives for this phase can be summarized in three points:

1. Users can independently complete the entire process from browsing products to placing an order and making payment.
 2. Operations staff can complete product maintenance, inventory management, order processing, and shipping operations in the backend.
 3. The system can form a basic data loop, providing a basis for subsequent optimization of metrics such as conversion rate, payment success rate, and order completion rate.in accordance with.
-

1.2 MVP Design Boundaries

As the first version of the MVP, this project should prioritize ensuring "the ability to buy, pay, ship, and track status." Therefore, the scope of this phase should revolve around the core transaction chain.

at presentCapabilities that should be prioritized for inclusion include:

Module	This issueKey Points	Product Judgment
Product Display	Product list, product details, SKU, price, inventory	Users must understand the product before they can make a purchase.
Shopping cart and order placement	Add to cart, buy now, submit order	This is the key step in converting browsing behavior into transaction behavior.
Payment	Initiating payment, payment result feedback, order status update	A successful payment signifies that the transaction is truly complete.
Order	Order list, order details, order status flow	Both users and operations need to clearly understand which stage an order is currently in.

Back-end operations	Product management, inventory management, order management, shipping management	Without backend support, frontend transactions cannot operate stably in the long term.
Basic data	Product browsing, adding to cart, placing orders, payment, order completion, and other metrics	After the MVP is launched, data is needed to determine whether the pipeline is functioning correctly.

Capabilities not prioritized for development in this phase include coupons, complex marketing campaigns, membership tiers, product reviews, personalized recommendations, live-streaming e-commerce, and social sharing. While these features can enhance growth and user experience, they are not essential for completing the basic transaction loop. Introducing complex marketing capabilities too early in the MVP phase may increase costs associated with price calculations, order rules, inventory management, and testing, ultimately impacting the efficiency of core system deployment.

1.3 User Role Definition

This system primarily involves three user roles: consumers, operations personnel, and platform administrators. Among them, consumers and operations personnel are the core roles in the MVP phase; platform administrators can retain basic permissions and exception handling capabilities in the first phase, and do not need to be overly complex.

User Role	Core Objectives	Main use cases	Key concerns
Consumer / Customer	Complete purchase	Browse products, view details, select SKUs, add to cart, submit order, pay, and check order status.	Clear product information, accurate inventory, smooth payment, and traceable orders.

Operations Staff	Supporting product sales and order fulfillment	Create products, maintain SKUs and inventory, view orders, arrange shipments, and process basic refunds.	Products can be quickly listed, inventory is controllable, and order processing is efficient.
Platform Admin	Ensure stable system operation	Manage backend account permissions, view platform data, and handle abnormal orders.	Access control is secure, data is visible, and anomalies are traceable.

From a product priority perspective, the consumer side determines whether a transaction occurs, while the operations side determines whether the transaction can be fulfilled. Therefore, the MVP phase should prioritize ensuring that consumers can make purchases smoothly, while ensuring that operations staff can support the flow of goods and orders. Platform administrators should take on more governance and fallback roles, and can start with a basic version to avoid excessive complexity in the backend during the first phase.

1.4 Core Consumer Scenarios

Consumers are the direct users of e-commerce systems and the source of transaction revenue. The core path for consumers after entering the platform typically includes: discovering products, understanding products, making a purchase decision, submitting an order, completing payment, and checking order status.

Scene	User behavior	System support
Browse products	Users can access the product list to view currently available products for purchase.	Product list, product images, product names, and price display.
View product details	Users click on the product to learn about its information.	Product details page, product images, price, SKU, inventory, sales volume

Select Specifications	Users select SKUs such as color, size, or model.	SKU selection, price linkage, and inventory linkage
add to the cart	The user will temporarily hold the item and settle the payment later.	Add to cart, modify quantity, delete item
Buy it now	Users proceed directly to the order confirmation process.	SKU verification, inventory verification, address confirmation, order amount calculation
Payment Orders	User completes online payment	Payment page, payment result feedback, order status update
View orders	Users can view order status and logistics information.	Order list, order details, shipping status, tracking number

The design focus on the consumer side is to reduce uncertainty during the purchase process. Users need to know clearly whether a product is available, its current price, whether there is sufficient stock, whether the order has been successfully paid for, and whether the product has been shipped. Therefore, in the MVP stage, even if the page visuals are not complex, it is essential to ensure that the product status, inventory status, and order status are clearly expressed.

1.5 Core Scenarios for Operations Personnel

Operations personnel are key to the continuous operation of an e-commerce system. Whether a consumer can successfully make a purchase depends not only on the front-end page, but also on whether the back-end products, inventory, and orders are properly maintained.

Scene	Operational behavior	System support
Create a product	Operations staff enter information such as product name, image, price, and category.	Product creation, product editing, and image uploading

Managing SKUs	Operations staff maintain prices and inventory for different specifications.	SKU creation, SKU pricing, SKU inventory
Products on shelves	Operations personnel control whether goods are available for sale.	Product status management: ON_SALE / OFF_SALE
Maintain inventory	Operations staff adjust product or SKU inventory	Inventory query, inventory modification, inventory verification
Processing orders	Operations staff review paid orders and arrange for fulfillment.	Order list, order details, order status
Arrange shipment	Operations staff fill in logistics information and update order status	Shipment management, tracking number entry, status update
Handling exceptions	Operations staff handle issues such as refunds, insufficient inventory, and order anomalies.	Refund processing, abnormal order inquiry, manual remarks

From a product design perspective, the operations backend should not be viewed merely as a "management page," but rather as a supporting system for the transaction process. Incomplete product information, incorrect SKU settings, and inaccurate inventory maintenance can all directly lead to failed orders or order fulfillment issues. Therefore, although backend functions do not directly interact with consumers, they are still core capabilities during the MVP (Minimum Viable Product) phase.

1.6 Core Scenarios for Platform Administrators

The platform administrator is primarily responsible for system governance and handling anomalies. During the MVP phase, the platform administrator is not a direct participant in the transaction process, so a complex management system is not required, but basic account permissions, data viewing, and anomaly handling capabilities should be available.

Scene	Management behavior	System support
Backend account management	Managing different back-end user accounts	User management, role management
Access Control	Limiting the scope of operations for different positions	Basic permission configuration
Data monitoring	View platform transaction and operational data	Core Data Dashboard
Exception handling	Inquire about and assist with abnormal orders.	Abnormal order query and status record

The platform administrator capabilities for this phase can be designed with a simplified approach, prioritizing the security of basic permissions and the traceability of anomalies. Complex permission matrices, audit logs, and risk control strategies can be gradually expanded as system transaction volume increases.

1.7 Role Relationships and Business Loop

These three types of roles together constitute the business loop of an e-commerce MVP: Consumers generate purchasing needs and complete browsing, ordering, and payment through the front end.

Operations staff are responsible for product listing, inventory maintenance, order processing, and shipping to ensure that orders are fulfilled.

The platform administrator is responsible for basic permissions, data monitoring, and anomaly handling to ensure the stable operation of the system.

From a business chain perspective, the consumer side is responsible for "generating orders," the operations side is responsible for "fulfilling orders," and the platform side is responsible for "ensuring system stability." Therefore, the subsequent functional priorities, process design, and PRD design of this solution will all revolve around this closed loop.

The product assessment at this stage is as follows: the first version of the MVP does not pursue full functionality, but rather the stability of the core transaction chain. As long as users can

successfully purchase goods, operations can correctly process orders, and the system can accurately record status and data, the MVP has the foundation for online validation. Subsequent updates will gradually expand marketing, recommendation, review, and refined operational capabilities based on real data.

II. MVP Feature List and Priority

2.1 Priority Definition

The initial goal of this project was to verify whether the basic e-commerce transaction loop could function successfully; therefore, the functional priorities needed to be determined based on the core process.

Browse products → Add to cart / Buy now → Submit order → Pay → Ship → Order complete

During the MVP phase, product managers need to control the scope of functionality to avoid overly complex first-version systems. Therefore, this approach uses three priority levels: P0, P1, and P2.

Priority	definition	Judgment criteria
P0	Must-be-launched features	Without this feature, users cannot complete the basic transaction process, or operations staff cannot support product sales and order fulfillment.
P1	Suggested features to be launched	It does not affect the basic transaction loop, but can significantly improve user experience, operational efficiency, or anomaly handling capabilities.
P2	Subsequent iteration features	The first version of the transaction loop is not required; it is primarily used for growth, marketing, retention, or refined operations.

From a product manager's perspective, P0 functions should answer "Can the system complete an order?", P1 functions should answer "Can the system complete orders more smoothly?", and P2 functions should answer more about "How can the system improve conversion and growth in the future?"

Therefore, the first version of an MVP should not aim for complete functionality, but should prioritize ensuring the stability and availability of key nodes such as products, inventory, orders, payments, and shipping. Otherwise, even if coupons, referral systems, or membership systems are added, if users ultimately cannot successfully pay or merchants cannot ship, the system has still failed to fulfill the core value of the MVP.

2.2 User-side function list and priority

The user-facing functionality primarily serves consumers, aiming to enable them to smoothly navigate the product browsing process to place an order and make a payment.

Module	Function	Priority	Product Judgment
Account System	Login / Register	P0	User identity needs to be linked to shopping cart, orders, payment records, and after-sales records, therefore it is a basic capability.
Product Center	Product List	P0	The product list is the entry point for users to discover products; without a product list, the purchase process cannot begin.
Product Center	Product details page	P0	Users need to view product images, names, prices, SKUs, and stock before making a purchase decision.
Product Center	Product status display	P0	Statuses such as "out of stock" and "sold out" must be clearly displayed to prevent users from submitting invalid orders.
SKU Selection	SKU Specification Selection	P0	Order and inventory deductions need to be linked to specific SKUs, especially applicable to products with multiple specifications such as color, size, and model.
shopping cart	add to the cart	P0	Supporting users to temporarily store goods and conduct unified settlement is an important part of the basic transaction chain.

shopping cart	Modify purchase quantity	P0	Users need to adjust the quantity before placing an order, and the system also needs to check the inventory based on the quantity.
shopping cart	Delete product	P1	Deleting items can improve the shopping cart experience, but it does not prevent users from completing a transaction by purchasing immediately.
Order System	Submit an order	P0	Submitting an order is the key step in transforming a shopping behavior into a transaction.
Order System	Order Amount Calculation	P0	The system must accurately calculate the price, quantity, and shipping costs of goods to avoid payment errors.
Address System	Shipping address entry / selection	P0	For physical goods to be shipped and fulfilled, a delivery address is required.
payment system	Online payment	P0	Payment is a crucial step in establishing a transaction; without the ability to pay, a valid order cannot be formed.
payment system	Payment result display	P0	Users need to know for sure whether the payment was successful, and the system also needs to update the order status synchronously.
Order System	Order List / Order Details	P0	Users need to view order status, product information, amount, address, and logistics progress.
Logistics system	View basic logistics information	P1	The MVP can initially display the shipping status and tracking number, without needing to integrate complex logistics tracking information.
Search Filters	Product Search	P1	Searching can improve search efficiency, but it's not absolutely necessary in the early stages when there are few products.
Search Filters	Category Filtering	P1	Category filtering helps users discover products, but it can be optimized after the basic product listing capabilities are completed.

After-sales system	Cancel order	P1	Supporting the cancellation of unpaid orders can reduce invalid orders and inventory holding.
After-sales system	Refund Request	P1	Refunds are important for user experience, but the MVP stage can first support a simplified refund process.
Marketing System	coupons	P2	Coupons can help with conversions, but they increase the complexity of price calculations and order rules, making them unsuitable for priority implementation in the first version.
User Interaction	Product Reviews	P2	Reviews can increase trust, but they don't affect the basic purchase loop; subsequent development is possible.
Recommendation system	Personalized recommendations	P2	Recommendations rely on user behavior data and algorithmic capabilities; however, in the early stages of an MVP (Minimum Viable Product), data is insufficient, making it a lower priority.

The core criterion for user-side P0 functionality is whether a user can complete a valid purchase. Therefore, product display, SKU selection, shopping cart, order submission, payment, and order viewing should be prioritized. While features such as search, refunds, and shipping details are important, they can be implemented in a simplified manner initially to avoid making the first version too broad.

2.3 List and Priority of Backend Operation Functions

The back-end operation functions mainly serve the operations staff, with the goal of supporting product listing, inventory maintenance, order processing, and delivery fulfillment.

Module	Function	Priority	Product Judgment
Product Management	Product creation	P0	Products must be created first before the front end has content to display and sell.
Product Management	Product Editing	P0	Operations staff need to maintain product names, images, descriptions, and prices to ensure the accuracy of product information.
Product Management	Product delisting and relisting	P0	Product status directly determines whether a user can purchase it, and therefore, product delisting control must be supported.
Product Management	Product Image Management	P0	Product images influence users' purchasing decisions and are an essential element of the product details page.
SKU Management	SKU creation	P0	For multi-specification products, SKUs are needed to differentiate specific product units such as color, size, and model.
SKU Management	SKU Price Maintenance	P0	Prices may vary for different SKUs; the order amount should be based on the price of the specific SKU.
Inventory Management	Inventory maintenance	P0	Inventory is crucial data for order verification and fulfillment, and its accuracy and controllability must be guaranteed.
Inventory Management	Inventory deduction / lock	P0	The system needs to process inventory during the order placement or payment stage to avoid overselling or invalid inventory usage.
Inventory Management	Inventory warning	P1	Inventory alerts can improve operational efficiency, but they are not a prerequisite for completing the first transaction.

Order Management	Order List	P0	Operations staff need to view all orders and process them accordingly.
Order Management	Order Details	P0	Operations staff need to view information such as products, users, addresses, amounts, and payment status.
Order Management	Order Status Management	P0	The system needs to support status transitions such as pending payment, paid, shipped, and completed.
Shipment Management	Create shipping information	P0	Orders that have been paid for must enter the shipment process, and operations personnel need to complete the fulfillment operation.
Shipment Management	Logistics tracking number entry	P0	The tracking number is used by users to check the shipment status and serves as proof of order fulfillment.
Shipment Management	Shipping status update	P0	After shipment, the order status should be updated from "paid" to "shipped".
After-sales management	Refund review	P1	Refunds fall under after-sales capabilities; in the MVP stage, basic verification and status updates can be supported first.
User Management	User account query	P1	It helps in handling order issues and customer service scenarios, but it is not essential for the core transaction loop.
Access Management	Backend role permissions	P1	This can reduce the risk of accidental operations. It is recommended to implement basic access control first in the MVP stage.
Data dashboard	Order and sales data statistics	P1	It helps in monitoring trading performance, but initially, it's best to focus on showcasing core indicators.

Marketing Management	Coupon Management	P2	Relying on complex marketing rules, it is not suitable to invest too many resources in the first version.
Content Management	Homepage Banner Management	P2	It helps with operational visibility but does not affect the basic transaction loop.
Refined Operations	User Tiered Management	P2	Suitable for building after the platform has a certain number of users and order volume.

From a product manager's perspective, backend operational capabilities cannot be simply viewed as "internal tools." For an e-commerce MVP, the backend directly determines whether the frontend has products to sell, whether inventory is accurate, and whether orders can be shipped on time. Therefore, product management, SKU management, inventory management, order management, and shipping management should all be placed in P0. However, the complexity of backend functions also needs to be controlled. For example, the first version can support single product creation and basic inventory maintenance, without necessarily requiring batch imports, complex approval workflows, inventory transfers, and advanced permission matrices. This ensures that the core operational processes run smoothly first, and then efficiency-oriented tools can be expanded according to the actual business scale.

2.4 Summary of MVP Core Functionality Scope

Based on the above priority criteria, the core scope of the first version of the MVP can be summarized as follows:

Business side	This issue must cover	Simplify coverage	Not included for the time being
User Purchase Path	Product list, product details, SKU selection, shopping cart, submit order, payment, order viewing	Product search, category filtering, logistics information, order cancellation, refund request	Coupons, reviews, favorites, recommendations, membership system

Operational fulfillment chain	Product creation, product editing, SKU management, inventory management, order management, and shipping management.	User inquiry, basic data dashboard, basic permission management, refund approval	Marketing campaigns, user segmentation, complex permissions, content operation positions
System data link	Order status, payment status, inventory status, product status	Basic indicator statistics and conversion funnel analysis	Advanced BI analytics, predictive models, personalized operations

The core goal of the first version of the MVP is not to cover all e-commerce capabilities, but to ensure that the following two links are valid:

User-side link:

Browse products → View details → Select SKU → Add to cart / Buy now → Submit order → Pay → View order

Operational side link:

Create product → Maintain SKU and inventory → Manage orders → Arrange shipment → Update order status

As long as these two links can operate stably, the system has the foundation to verify business value from scratch.

2.5 Description of features not included in this MVP

To control the scope of the first version development, the following features are recommended not to be included in the MVP, but to be planned for subsequent versions.

Function	Reasons for not including	Subsequent plans
Coupon system	This will increase the complexity of discount rules, price calculation, order amount verification, and refund processing.	It can be built as a marketing module after the payment chain is stable.
Membership System	This involves points, levels, benefits, and growth rules, but does not affect the basic purchase process.	It can be built after the user base grows.

Product Reviews	It helps build trust, but it's not a prerequisite for completing the first transaction.	A review section can be added after the order is completed.
Personalized recommendations	Relying on user behavior data and recommendation algorithms, MVP initially suffers from insufficient data.	It can be built after accumulating browsing, add-to-cart, and purchase data.
Marketing campaign	The development and testing costs are high, involving event pricing, event inventory, event pages, and event rules.	Can be used as a growth version plan
Live streaming e-commerce	This is not considered a basic e-commerce transaction capability; it requires a higher level of content operation and technical skills.	Not considering at the moment
Social sharing	It can enhance reach, but it does not affect users' ability to complete a purchase.	Further increases will be made based on growth demand.

From a product manager's perspective, omitting these features doesn't mean they're unimportant; rather, it's because they're not among the most critical validation objectives of the first phase. The MVP phase should prioritize addressing the core question of "whether users can buy the product," rather than prematurely pursuing "why users buy more, repeat purchases, and spread the word." Adding complex marketing features before the basic transaction chain is stable may actually increase order amount errors, abnormal inventory usage, and the complexity of after-sales processing.

2.6 Summary of Functional Priority Design

The core logic behind this MVP feature priority design is: first ensure the transaction loop, then improve the experience and efficiency, and finally consider growth and refined operations.

P0 The main functions revolve around products, SKUs, inventory, orders, payments, and shipping, with the aim of ensuring that users can complete an order and operations staff can fulfill a contract.

P1 The features are primarily designed to improve user experience and operational efficiency, such as search filtering, logistics display, refund processing, data dashboards, and basic permissions.

P2 The features primarily support future growth, such as coupons, reviews, memberships, referrals, and marketing campaigns.

Therefore, the product assessment for the first version of the MVP can be summarized as follows:

First, make the system usable; then, make the processes easy to use; and finally, let the business grow.

This prioritization helps the team focus resources on completing core functions within a limited timeframe, reducing the complexity of the first version of the system while maintaining a clear direction for subsequent iterations.

III. Core Process Design

This section selects two core processes for detailed design:

1. User order placement and payment process
2. Product creation and publishing process

The reason for choosing these two processes is that the user order placement and payment process is the core transaction link of the e-commerce system, which directly determines whether the user can complete the purchase; the product creation and publishing process is the prerequisite for the transaction link. Only when the product information, SKU, inventory and product status are configured correctly can the front-end purchase process run normally.

From a product manager's perspective, the process design of an e-commerce MVP cannot only describe "how things should go under ideal conditions," but must also cover abnormal scenarios. This is because in real-world business scenarios, insufficient inventory, payment failures, payment timeouts, product delisting, and SKU invalidation are very common. If these anomalies are not clearly defined in advance, they will directly impact user experience, order fulfillment, and development implementation.

3.1 User order placement and payment process

3.1.1 Process Objectives

The goal of the user order and payment process is to support consumers from selecting products to completing payment and generating a valid order.

This process needs to ensure that:

1. The product is available for purchase.
2. The SKU selected by the user is valid;
3. Sufficient inventory;
4. The order amount is calculated accurately;
5. Payment results are correctly synchronized;
6. Order status can be changed according to the rules.

This process is the core link of an e-commerce system's MVP. If a user cannot successfully submit an order or complete payment, even if the product display and backend management functions are complete, the system cannot verify the true value of the transaction.

3.1.2 Roles Involved

Role	illustrate
consumer	Select products, confirm order, and complete payment.
system	Verify product status, SKU, inventory, price, address, and payment result.
payment platform	Process payment requests and return a status indicating payment success, failure, or processing.
Operations personnel	Arrange shipment and fulfillment after successful order payment.

3.1.3 Prerequisites

Before a user can proceed with the order process, the following conditions must be met:

1. The user is logged in;
2. Product status is **ON_SALE**;
3. The user has selected a valid SKU;
4. Purchase quantity greater than 0;
5. SKU inventory is greater than or equal to the purchase quantity;
6. The user has filled in or selected a valid shipping address;
7. The system can read the latest product prices and inventory information.

If the above conditions are not met, the system should prevent the user from submitting the order and provide a clear prompt.

3.1.4 Normal Procedure

step	User actions / System actions	illustrate
1	User enters product details page	View product images, name, price, SKU, inventory, and other information.
2	User selects SKU	For example, color, size, model, etc.
3	User selects purchase quantity	The system determines the validity of the quantity based on the SKU inventory.
4	Users click "Buy Now" or checkout from their shopping cart.	Entering the order confirmation page
5	System checks product status	Determine if the product is still in ON_SALE
6	System verification of SKUs and inventory	Determine if the SKU is valid and if the inventory is sufficient.
7	User confirms shipping address	The address will be used for subsequent shipping.
8	The system calculates the order amount.	This includes the product price, quantity purchased, and shipping costs. Coupons will not be available during the MVP phase.
9	The user clicks "Submit Order"	System creates order

10	The system generates an order number.	The order status has changed to PENDING_PAYMENT
11	The user enters the payment page	Initiate payment
12	The payment platform returns the payment result.	Possible results: success, failure, or in progress.
13	Payment successful	The system will update the order status to PAID.
14	Users view payment results	The page displays a successful payment message.
15	Order enters the pending shipment process	The operations staff will arrange subsequent shipments.

3.1.5 Order Status Changes

Business Node	Order status	illustrate
User successfully submitted order.	PENDING_PAYMENT	The order has been generated, but the user has not yet paid.
User payment successful	PAID	The payment platform has confirmed successful payment, and the order has entered the pending shipment stage.
Merchant shipping	SHIPPED	The operations staff fills in the logistics information and confirms the shipment.
User confirms receipt / System automatically confirms receipt	COMPLETED	Transaction completed
The user failed to pay within the specified time.	CANCELLED	The system automatically canceled the order.
User cancels unpaid order	CANCELLED	User cancels order
User requests refund after payment	REFUNDING	The order is in the process of being refunded.

Refund completed

REFUNDED

Refund completed

3.1.6 Inventory Change Rules

Inventory management is a key design element in the order placement process. The MVP phase recommends the following: **Inventory is locked when an order is submitted, officially deducted after successful payment, and released after payment timeout or order cancellation.**

Business Node	Inventory handling methods	illustrate
Users browse products	No deduction of inventory	Browsing behavior does not represent purchase intention
Add to cart by user	No deduction of inventory	Shopping carts are only for temporarily storing goods and should not occupy inventory.
User submits order	Lock in inventory	To prevent overselling caused by multiple users purchasing at the same time
User payment successful	Officially deduct inventory	Once payment is successful, the order becomes a valid transaction.
User payment timed out	Release locked inventory	To avoid inventory being tied up in invalid orders for an extended period
User cancels unpaid order	Release locked inventory	Order invalid, inventory restored
Refund after payment	Whether to replenish inventory depends on the shipment status.	Refunds for unshipped items can usually replenish inventory; refunds for shipped items need to be processed based on the return results.

From a product manager's perspective, inventory deduction strategies need to balance "reducing the risk of overselling" and "controlling system complexity." For the first version of the MVP, if the order volume is low, a simplified solution of deducting inventory after successful payment can be adopted; however, from a more robust design perspective, locking inventory

when submitting an order is more suitable to avoid the problem of multiple people purchasing the same inventory at the same time.

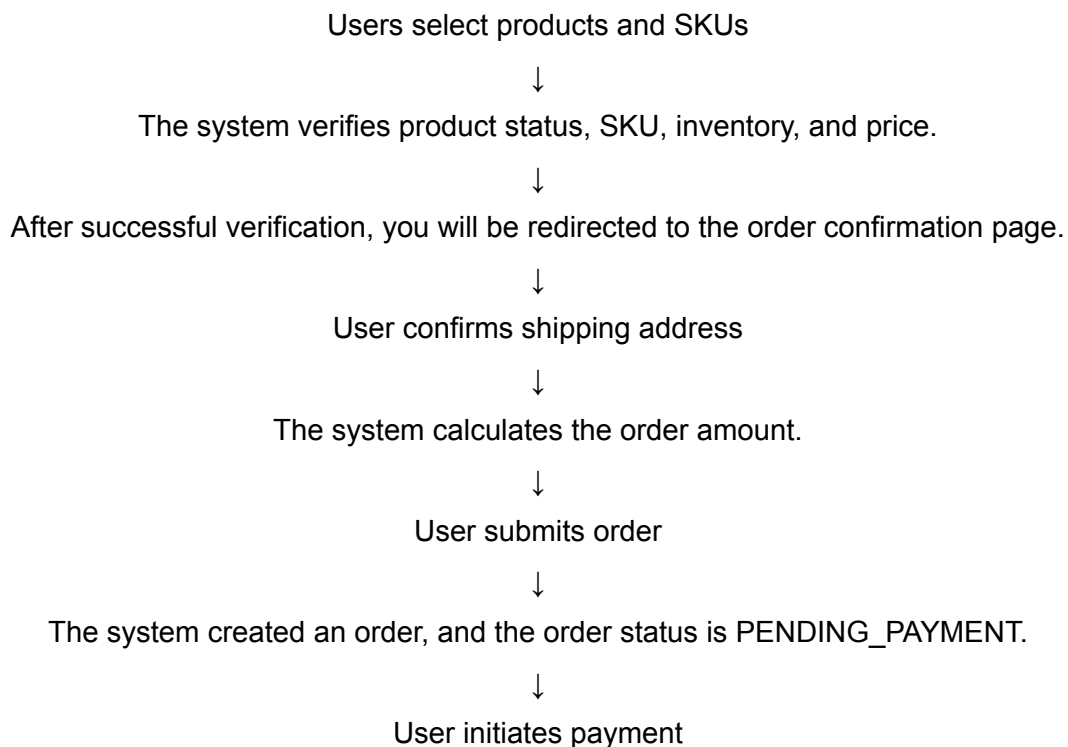
3.1.7 Abnormal Scenario Design

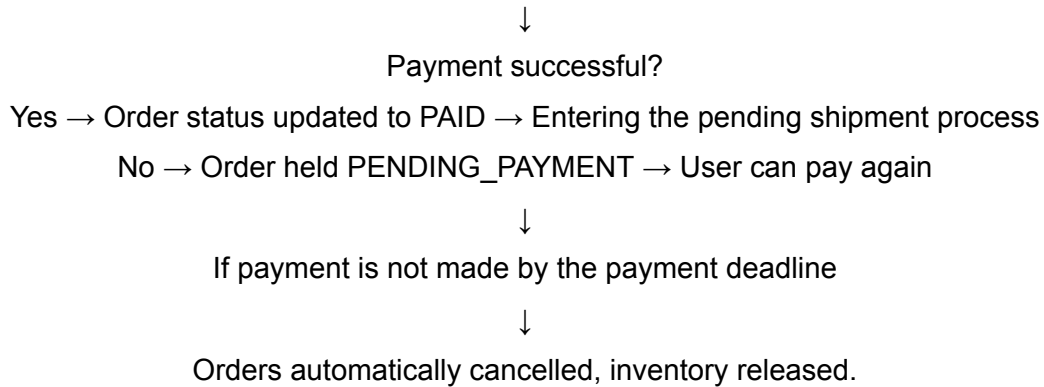
Abnormal scenarios	Triggering conditions	System processing	Page prompts
Insufficient inventory	When placing an order, the SKU inventory is less than the purchase quantity.	Prevent order creation and avoid proceeding to payment.	The current stock is insufficient. Please adjust the quantity you wish to purchase or select another specification.
The product has been removed from the shelves.	The item was in the shopping cart, but was removed from the shelves before checkout.	The product is marked as unavailable and cannot be selected for checkout.	This product has been removed from the shelves and is temporarily unavailable for purchase.
SKU expired	The user-selected SKU was deleted or disabled.	Prevent order submission and guide users to make a new selection.	The current product specifications are no longer valid. Please select a different one.
Price changes	After a user browses the product or SKU, the price may change.	Prevent orders from being submitted at the old price and require user reconfirmation.	The product price has been updated, please confirm again.
Payment failed	The payment platform returned a failure result.	Orders held PENDING_PAYMENT Allow repayment	Payment failed. Please try again or change your payment method.

Payment timeout	The user failed to pay within the specified time.	Order status changed CANCELLED and release inventory	The order payment timed out and has been automatically cancelled. Please place a new order.
Payment callback delay	The user has made the payment, but the system has not yet received a payment success notification.	The page displays "Payment processing in progress," and the system actively queries the payment result.	Payment confirmation in progress, please check your order status later.
Invalid address	The user did not fill in an address or the address was incomplete.	Prevent order submission	Please fill in a valid shipping address.

3.1.8 Flowchart Description

The user order placement and payment process can be described as follows:





3.1.9 Product Manager Judgment

In this process, I will focus on three product design principles.

1. Backend validation must be used as the final basis for judgment. Inventory, price, and product status in the product details page and shopping cart may be cached or delayed. Therefore, when submitting an order, the backend must re-validate the product status, SKU, inventory, price, and address information.
 2. Payment failures must be recoverable. Payment failures should not directly result in the user losing their order, but should allow the user to pay again; the payment process should not immediately determine failure, but the final status should be confirmed through payment result queries or callback mechanisms.
 3. Order status must be clear and traceable. Users, operations staff, and the system should all see the same order status; otherwise, issues such as users paying but the backend not shipping the goods, or the backend shipping the goods but users not seeing the logistics information, will arise. These kinds of problems directly affect user trust.
-

3.2 Product Creation and Shipping Process

3.2.1 Process Objectives

The goal of the product creation and publishing process is to support operations staff in creating products in the backend and publishing them to the frontend for consumers to browse and purchase.

This process needs to ensure that:

1. Complete basic product information

2. Product images and descriptions can be used for front-end display;
3. Product price valid
4. SKU information is correct
5. Accurate inventory quantity
6. Product status is controllable
7. The listed products can be displayed normally on the front end.

Product creation and publishing are the initial steps in the e-commerce transaction process.

Only when product information is accurate, inventory is available for sale, and the product status is normal can the user's purchase process have a foundation.

3.2.2 Roles Involved

Role	illustrate
Operations personnel	Create products, set SKUs, maintain prices and inventory, and execute put-away or delisting.
system	Validate product fields, save product information, control product status, and synchronize front-end display.
consumer	Browse and purchase listed products on the front end.
Platform Administrator	Handle product permissions or abnormal products when necessary.

3.2.3 Definition of Product Status

Product status	meaning	Front-end performance
DRAFT	Draft, product information incomplete	Invisible to the front end
ON_SALE	The product is now available for purchase.	Visible and available for purchase on the front end
OFF_SALE	The product has been removed from the	Visible or invisible on the front end, but cannot be purchased.

shelves and is no longer
available for sale.

SOLD_OUT Sold out, 0 in stock. The front end is visible, but the
"Buy" button is unavailable.

If a product review mechanism is not designed for the MVP stage, it can be omitted. **PENDING_REVIEW** To avoid increasing process complexity, operations staff can save a product as a draft after creation and then manually upload it.

3.2.4 Normal Procedure

step	Operation / System Actions	illustrate
1	Operations personnel access the backend product management page	View product list
2	Click "Create New Product"	Enter the product creation page
3	Fill in the basic product information	Including product name, category, images, description, etc.
4	Set product price	Fill in the selling price and original price
5	Set SKU information	Such as color, size, model, etc.
6	Set SKU price and inventory	Different SKUs may have different prices and inventory levels.
7	System verifies product information	Validate the completeness of required fields, price, inventory, and SKU.
8	Operations personnel store goods	The product status is DRAFT.
9	The operations staff clicked "List".	The system will re-verify whether the product meets the conditions for sale.
10	The system updates the product status to ON_SALE.	The product is now available for sale.
11	Front desk product display	Users can browse and purchase

3.2.5 State Changes

Business Node	Product status	illustrate
Created a product but not fully filled in	DRAFT	Products are only visible in the backend.
Fill in the product information completely and save.	DRAFT	The product has not yet been officially launched.
The product passes verification and is clicked to be listed.	ON_SALE	The product is visible and available for purchase on the front end.
The operators took the initiative to remove it.	OFF_SALE	The product is unavailable for purchase.
The inventory of goods is 0.	SOLD_OUT	Products can be displayed but not purchased.
Operations to replenish inventory and restock	ON_SALE	The product is now available for purchase.

3.2.7 Product Field Design

When creating a product, operations staff need to fill in the following fields:

Field	Is this field required?	illustrate
Product Name	yes	Used for front-end display and search

Product Categories	yes	Used for product categorization, filtering, and backend management.
Product Images	yes	Used for displaying product lists and detail pages
Product Description	yes	Help users understand the selling points of the product
Selling price	yes	Actual purchase price for users
Original price	no	Used for price comparison display
SKU Name	yes	Such as white, black, size M, size L, etc.
SKU Price	yes	Different SKUs may have different prices.
SKU Inventory	yes	Used for order verification and inventory deduction
Product status	yes	Controlling whether products are visible and available for sale
Shipping rules	MVP is optional	Used for order amount calculation
After-sales rules	MVP is optional	For refunds and after-sales instructions

3.2.8 Business Rules

rule	illustrate	Error message
Product name cannot be empty	The product name is the basic information that users use to identify products.	Product name cannot be empty
Upload at least one product image.	Images can influence users' purchasing decisions.	Please upload at least one product image.
The selling price must be greater than 0.	The price must be a valid amount.	The selling price of the product must be greater than 0.
SKU inventory cannot be less than 0	Inventory cannot be negative.	SKU inventory cannot be less than 0
Each product must have at least one valid SKU.	Orders need to be linked to specific SKUs	Please set at least one valid SKU before listing.
Integrity verification must be passed before listing.	Products with incomplete information cannot be sold on the front end.	Product information is incomplete and cannot be listed at this time.
Products that have been removed from the shelves are no longer available for purchase.	OFF_SALE items cannot be added to the cart or purchased immediately.	This product has been removed from the shelves and is temporarily unavailable for purchase.
You cannot purchase when the stock is 0.	SOLD_OUT items or SKUs are not allowed to be submitted for order.	Current inventory is insufficient

3.2.9 Abnormal Scenario Design

Abnormal scenarios	Triggering conditions	System processing	State change
Product information is incomplete.	The required fields were missing when the operations staff clicked "list".	Prevent listing and mark missing fields.	DRAFT → DRAFT

SKU inventory is 0	A certain SKU has 0 units in stock.	This SKU is unavailable for purchase; if all SKUs are 0, the product is sold out.	ON_SALE → SOLD_OUT
abnormal commodity prices	The selling price is 0, a negative number, or the SKU price is empty.	Prevent saving or listing, prompting you to adjust the price.	DRAFT → DRAFT
The product was removed from the platform by the operations team.	The product was listed, but the operations team proactively removed it.	Purchases are disabled on the front end; the shopping cart label is invalid.	ON_SALE → OFF_SALE
Sold out	All SKU inventory becomes 0.	The "Buy" button is grayed out, indicating that the product is sold out.	ON_SALE → SOLD_OUT
Replenish inventory	Operations staff replenish inventory for sold-out items.	The product can be relisted for sale.	SOLD_OUT → ON_SALE

3.2.10 Flowchart Description

The product creation and publishing process can be described as follows:

Operations personnel access the backend product management



Click to create a new product



Fill in the product name, category, image and description.



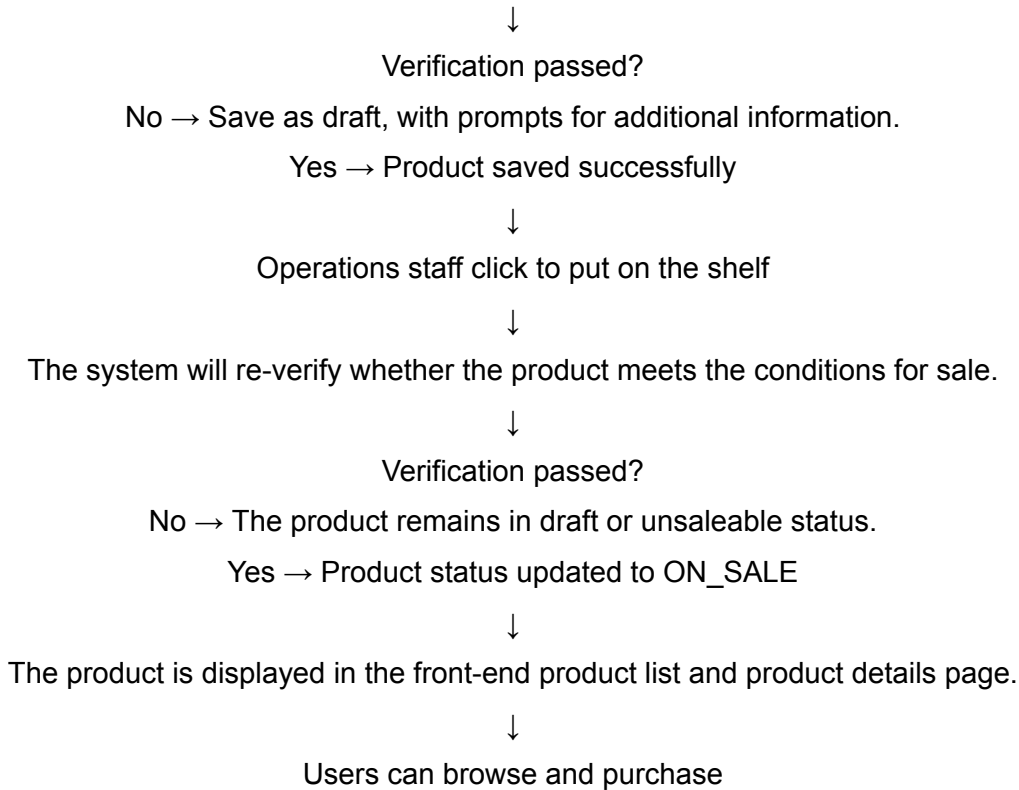
Set SKU, price, and inventory.



Click Save



The system performs field integrity verification.



3.2.11 Product Manager's Judgment

In the product creation and publishing process, I will focus on three key issues.

1. Product status must be simple and clear. Complex approval workflows are unnecessary in the MVP phase; avoid making the first version of the backend too cumbersome. **DRAFT**, **ON_SALE**, **OFF_SALE** and **SOLD_OUT** It is already able to cover the basic product lifecycle.
2. SKUs should be the core unit for inventory and pricing management. Users ultimately purchase specific SKUs, not abstract products. For example, for the same pair of headphones, the white SKU and the black SKU may have different prices and different stock levels. Therefore, order submission, inventory deduction, and price display should all be centered around the SKU.
3. Listing verification is more stringent than saving verification. Operations staff can save incomplete products as drafts, but once a product is listed, it must meet basic requirements such as product name, image, price, SKU, and inventory. This prevents incomplete products from appearing on the front end and affecting users' purchasing decisions.

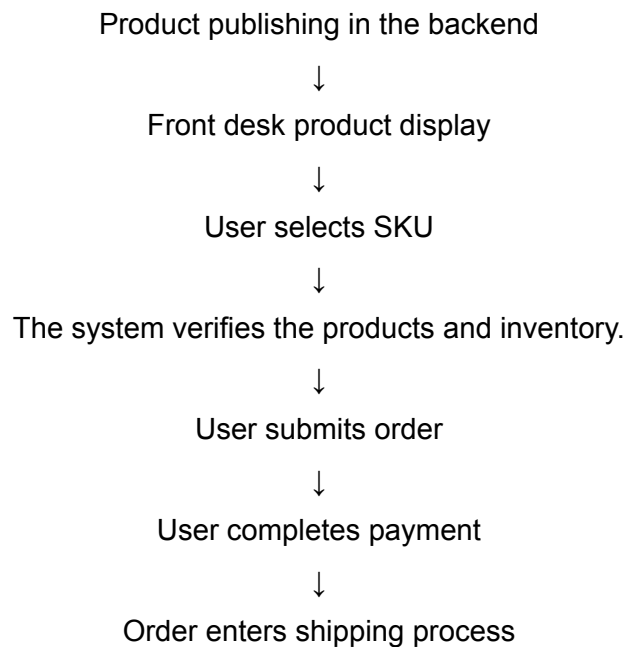
3.3 Relationship between the two processes

The user order and payment process depends on the product creation and publishing process.

A user can only make a purchase if the product in the backend meets the following conditions:

1. The product has been created;
2. Product status is **ON_SALE**;
3. The product must have at least one valid SKU;
4. SKU inventory is greater than 0;
5. The product price is valid;
6. Product information can be displayed normally on the front end.

The relationship between the two processes can be understood as follows:



Therefore, the product management process determines "whether there are products to sell," while the order placement and payment process determines "whether the user can complete the purchase." Together, they constitute the core business foundation of an e-commerce MVP. From a product manager's perspective, the initial MVP design should avoid overly pursuing complex features and instead prioritize ensuring that key states and key validations are clear

and reliable. As long as product status, SKU inventory, order status, and payment status can flow correctly, the system has the foundation for online verification.

IV. Simplify PRD Design: Product Details Page

4.1 Function Name

Product Detail Page

4.2 Functional Background

The product details page is a key page in an e-commerce system that connects "product browsing" and "order placement".

After seeing a product in the product list, users need to go to the product details page to learn more about the product, including product images, product name, price, SKU specifications, inventory, sales volume, etc. Only after fully understanding the product information will users decide whether to add it to their cart or make an immediate purchase.

Therefore, the product detail page not only serves the function of displaying products, but also guides purchase decisions, making it an important page that affects user conversion rates.

In this MVP version, the core goal of the product details page is not to achieve complex marketing displays, but to ensure that users can clearly understand the product information and smoothly enter the subsequent purchase process.

This feature needs to address the following key issues:

1. Can users clearly see what the product is?
2. Can users know the current price of a product?
3. Can the user select the correct SKU?
4. Can users know whether the current product or SKU is in stock?
5. Can users add items to their cart or purchase them immediately when the items are available for sale?
6. Can the system provide a clear prompt when a product is taken off the shelves, sold out, or in short supply?

From a product manager's perspective, the product details page is a user-side P0 function because if users cannot view the product details, they cannot make a purchase decision, and the subsequent shopping cart, order submission, and payment processes cannot proceed smoothly.

4.3 User Goals and Usage Scenarios

The target users are consumers (Customer / Buyer).

The core objectives for users after entering the product details page include:

1. Understand the basic information about the product;
2. View product images;
3. View the current selling price and original price;
4. Select product specifications, such as color, size, model, etc.;
5. Check the inventory of the selected SKU;
6. Determine whether a product is worth buying;
7. Add items to your shopping cart;
8. Proceed directly to the order placement process.

Use cases	User behavior	System support
Access the product details page from the product list.	User clicks on product card	The system loads product details based on productId.
View product information	Users view information such as images, names, prices, and sales volume.	The page displays basic product information.
Select SKU	User selects color, size, or model	The page dynamically updates SKU prices and inventory.
add to the cart	The user will not purchase at this time; the item will be saved.	The system adds the item to the cart after verifying login, SKU, and inventory.
Buy it now	Users directly enter the order placement process	The system verifies the product status, SKU, inventory, and quantity.

The product is unavailable for purchase.

The product is unavailable, sold out, or the current SKU is out of stock.

The purchase button is grayed out on the page, and the reason is displayed.

4.4 Page Structure

Product detail pages are recommended to include the following areas:

1. Top navigation area: Back button, page title;
2. Product Image Section: Main product image, image thumbnails;
3. Product information area: Product name, selling price, original price, sales volume;
4. SKU Selection Area: Specification Options, Current SKU Inventory;
5. Quantity selection area: Decrease, Quantity Input, Increase;
6. Operation button area: Add to cart, Buy now;
7. Product Description Section: Product description, delivery instructions, or after-sales instructions.

The page structure can be referenced from the low-fidelity prototype:

图4-1 产品详情页线框图
Product Detail Page Wireframe

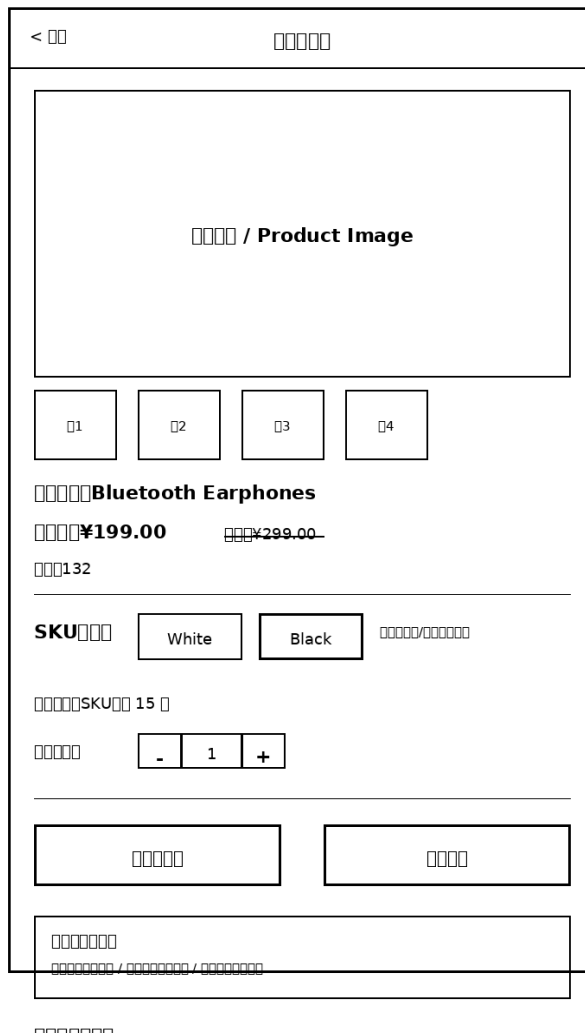


Figure 4-1 Low-fidelity prototype of the product details page

From an MVP perspective, this page doesn't need to be cluttered with excessive marketing information. The focus should be on basic product information, SKU selection, inventory status, and the purchase entry point. In particular, SKUs and inventory must be clearly displayed; otherwise, users might easily see that a product is in stock but be unable to purchase it after selecting a specific size.

4.5 Page Field Design

area	Field	Field Description	Is it necessary?
Product Image Area	Product main image	Displaying key product images helps users quickly identify the product.	yes
Product Image Area	Product Image List	Displays multiple product images, allowing users to view more details.	no
Product Information Area	Product Name	Display product names, such as Bluetooth Earphones	yes
Product Information Area	Product ID	Used for system product identification; may not be displayed on the front end.	yes
Product Information Area	Product status	Determine whether a product is listed, for example, ON_SALE / OFF_SALE.	yes
Price Zone	Selling price	The actual price paid by the user for the goods	yes
Price Zone	Original price	Used for price comparison displays to help users understand discounts.	no
Sales data area	Sales	Displaying cumulative sales volume enhances user trust.	no
SKU area	SKU Name	Display specification options, such as white and black.	yes
SKU area	SKU Price	Different SKUs may have different prices.	yes
SKU area	SKU inventory	Current SKU available quantity	yes
Inventory Area	Total inventory	Overall inventory of goods, used to determine overall sell-out status.	yes
Quantity area	Purchase Quantity	The quantity the user plans to purchase this time	yes
Operating area	Add to cart button	Users add items to their shopping cart	yes
Operating area	Buy Now button	Users are directly taken to the order confirmation page.	yes

Prompt Area	Error message	Display abnormal information such as unavailable, yes sold out, and insufficient stock.
-------------	---------------	---

4.6 Core Interaction Design

4.6.1 Accessing the product details page

After a user clicks on a product from the product list, the system requests the product details API based on the product ID and displays the product information.

If the API returns a successful response, the page will display information such as product name, image, price, SKU, and inventory.

If the API fails, an error message will be displayed on the page.

Prompt text: [Product information failed to load, please try again later.](#)

4.6.2 SKU Selection

If a product has multiple SKUs, users need to select the SKU before adding it to their cart or purchasing it immediately.

For example:

Product: Bluetooth Earphones

SKU 1: White, Price: ¥199.00, Stock: 10

SKU 2: Black, Price ¥209.00, Stock 15

When a user selects a different SKU, the page needs to be updated accordingly:

1. Current displayed price;
2. Current SKU inventory;
3. Purchase button status;
4. The maximum number of items a user can purchase.

If a user clicks "Add to Cart" or "Buy Now" without selecting an SKU, the system should prompt the user to select the specifications first.

Prompt text: [Please select the product specifications first.](#)

4.6.3 Selection of Purchase Quantity

Users can adjust the purchase quantity using the quantity selector.

Purchase quantity rules:

1. The default purchase quantity is 1;
2. The quantity purchased must be greater than 0;
3. The quantity purchased cannot exceed the current SKU inventory;
4. If the current SKU inventory is 0, the quantity selector is not operable;
5. If the user enters an invalid quantity, the system should automatically correct it or display an error message.

Prompt text: [The quantity purchased cannot exceed the current inventory.](#)

4.6.4 Add to Cart

When a user clicks "Add to Cart," the system needs to verify:

1. Is the user already logged in?
2. Is the product currently listed?
3. Has the user selected an SKU?
4. Is the current SKU in stock?
5. Is the quantity purchased legal?
6. Once the verification is successful, the item will be added to your shopping cart.

Success message: [Added to cart.](#)

If the user is not logged in, you can guide them to log in first.

Prompt text: [Please log in before adding to your cart.](#)

4.6.5 Buy Now

When a user clicks "Buy Now", the system needs to verify:

1. Is the user already logged in?
2. Is the product in a state of...[ON_SALE](#);
3. Has the SKU been selected?
4. Are there sufficient SKUs in stock?
5. Is the quantity purchased legal?
6. After successful verification, the user is directed to the order confirmation page.

After successful verification, the user is directed to the order confirmation page.

If the verification fails, the user will remain on the product details page with the corresponding error message displayed.

4.7 Business Rules

rule	illustrate	Page presentation / tips
Goods must be in ON_SALE Only then are purchases allowed.	OFF_SALE The product is unavailable for purchase.	The "Add to Cart" and "Buy Now" buttons are grayed out, and a message appears stating, "This item is no longer available and cannot be purchased at this time."
Users must select an SKU before they can make a purchase.	Orders need to be linked to specific SKUs for pricing, inventory, and shipping.	The prompt reads, "Please select product specifications first."
The price on the page should change dynamically with each SKU.	Different SKUs may have different prices.	Selecting the Black SKU will display ¥209.00
Inventory display should be based on the current SKU inventory.	Total inventory cannot replace specific SKU inventory.	Selecting an SKU will display the inventory for that SKU.
You cannot purchase when the stock is 0.	Orders cannot be submitted if the product or SKU is out of stock.	The "Buy" button is grayed out, and the message reads "Sold Out".
Adding to cart does not lock inventory	Adding items to your cart only indicates an intention to purchase; it does not mean the transaction has been completed.	Inventory should be processed during the order submission or payment stage.

Orders must be re-verified before submission.	Front-end data may be cached or expired.	The backend re-verifies the product status, SKU, price, and inventory.
Price display requires conversion of amount format	In the API, price = 19900 usually represents 199.00 yuan.	The page displays ¥199.00 instead of ¥19900.

From a product manager's perspective, the most common problem with product detail pages isn't whether the page can be displayed at all, but rather whether the displayed information can support the user in making the correct purchase decision. Therefore, price, SKU, and inventory must be linked, and the final result must be verified by the backend before order submission.。

4.8 Exception Handling and Error Messages

Abnormal scenarios	Triggering conditions	Page performance	Prompt text
Product information failed to load	Product details API request failed	The page displays a loading failure status.	Product information failed to load, please try again later.
The product has been removed from the shelves.	Product status is OFF_SALE	Gray out the purchase button	This product has been removed from the shelves and is temporarily unavailable for purchase.
Sold out	Total inventory is 0.	The "Buy" button is grayed out, indicating that the product is sold out.	The product is currently sold out.
SKU inventory insufficient	The user-selected SKU has 0 units in stock.	This SKU is currently unavailable for purchase.	Current specifications are in short supply.

No SKU selected	The user clicked to buy without selecting an SKU.	Prevent operation	Please select the product specifications first.
Purchase quantity exceeds inventory	<code>buyCount > skuStock</code>	Prevent operation	The quantity purchased cannot exceed the current inventory.
User not logged in	Users who are not logged in click to add to cart or purchase	Guide to Login	Please log in before proceeding.
Price changes	Prices differ before and after placing the order	Prevent ordering at the original price	The product price has been updated, please confirm again.
Network error	Request timed out or network failed	Keep the current page open and allow retries.	Network error, please try again later.

Error messages should be user-friendly rather than directly displaying system fields. For example, it is not recommended to directly display "`OFF_SALE`" or "`Insufficient inventory`". Instead of "This product is no longer available," it should be changed to "This item is out of stock" or "Current specifications are out of stock." Users are not API documentation readers, although product managers are sometimes forced to pretend that everyone loves reading error codes.

4.9 Acceptance Criteria

4.9.1 Page Display Acceptance

1. After entering the product details page, users can see the product name, product image, price, sales volume, SKU, and inventory information.
2. Product images load correctly.
3. The product price is displayed in the correct format, for example, ¥199.00.
4. If an original price exists, the page should display that price.
5. If sales figures exist, the page should display those figures.

4.9.2 SKU Interactive Acceptance

1. The page can display all available SKUs.
2. When a user selects different SKUs, the price on the page changes correctly.
3. After a user selects different SKUs, the inventory on the page updates correctly.
4. SKUs with zero stock cannot be purchased.
5. If a user clicks to buy without selecting an SKU, the system prompts "Please select product specifications first".

4.9.3 Inventory Rule Acceptance

1. When the total inventory of goods is 0, the purchase button is grayed out.
2. When the current SKU inventory is 0, the purchase button is grayed out.
3. The system blocks the operation when a user purchases more than the stock is available.
4. The inventory quantity displayed on the page is consistent with the inventory returned by the API.
5. Before submitting an order, the system will re-verify the inventory.

4.9.4 Goods Condition Inspection

1. When the product status is **ON_SALE** At that time, users can make purchases normally.
2. When the product status is **OFF_SALE** At that time, the page displayed "Product has been removed from shelves".
3. **OFF_SALE** The "Add to Cart" and "Buy Now" buttons for the product are not clickable.
4. If a product is removed from the shopping cart, the user cannot submit an order for that product at checkout.

4.9.5 Add to cart and check

1. After selecting the SKU and quantity, the user can successfully add the item to their shopping cart.
2. After successfully adding the item to your cart, a success message will be displayed on the page.
3. When an unlogged-in user clicks "Add to Cart," the system will guide them to log in.
4. You cannot add items to your cart if they are unavailable or out of stock.

4.9.6 Immediate Purchase and Acceptance

1. After selecting the SKU and quantity, users can click "Buy Now".
 2. After clicking "Buy Now", the system will redirect you to the order confirmation page.
 3. You cannot proceed to the order confirmation page if the product is unavailable, out of stock, or if no SKU is selected.
 4. Before redirecting to the order confirmation page, the system should verify the product status, SKU status, inventory, and price.
-

4.10 Data Tracking and Recording

Although the question does not mandate the use of tracking data, from a product manager's perspective, the product details page is a crucial node in the conversion process, so it is recommended to record the following basic data.

Event tracking	Triggering timing	use
product_detail_view	User enters product details page	Count the number of visits to the product details page
sku_select	User selects SKU	Specifications for analyzing user preferences
add_to_cart_click	The user clicks "Add to Cart"	Analysis of purchase intentions
buy_now_click	The user clicked "Buy Now"	Analyze purchase intentions
add_to_cart_success	Added to cart successfully	Statistics on the success rate of adding items to cart
buy_now_success	Successfully entered the order confirmation page	Analyze the conversion rate from the product details page to the order page.

product_unavailable_show	Display notifications for items being unavailable or sold out.	Monitoring the impact of unpurchaseable items on conversion rates
--------------------------	--	---

This data can help product managers analyze product detail page views, add-to-cart rates, purchase conversion rates, and the impact of inventory anomalies on user behavior.

4.11 Relationship with other modules

Related modules	Relationship Explanation
Product Management Module	The backend maintains information such as product name, images, price, and status, while the frontend details page reads and displays this information.
SKU Management Module	The product detail page displays SKU options, and shows prices and stock levels based on the SKU.
Inventory Management Module	The product details page displays the inventory; inventory verification is required before placing an order.
Shopping cart module	After the user clicks "Add to Cart," the product, SKU, and quantity are added to the cart.
Order module	After the user clicks "Buy Now", they will proceed to the order confirmation and order submission process.
User Module	Check if the user is logged in; if not, the user needs to log in first.
Payment module	The product details page does not directly process payment, but it guides users through the subsequent order payment process.

4.12 Scope of this MVP and subsequent iterations

MVP Implementation Scope for This Period

This product details page primarily implements the following capabilities:

1. Display basic product information;
2. Display product images;
3. Displays the selling price and original price of the product;
4. Display SKU list;
5. Supports SKU selection;
6. Display prices and inventory based on SKU;
7. Add to cart;
8. Supports immediate purchase;
9. Supports product delisting and low inventory alerts.

Subsequent iterative directions

Subsequent versions may add the following capabilities:

1. Product reviews are displayed.
2. Rich text description of product details;
3. Product parameter table;
4. Product recommendations;
5. Coupon redemption;
6. Collect items;
7. Share products;
8. Arrival notification;
9. Recommended similar products.

These capabilities can help improve conversion rates and user experience, but they do not affect the basic transaction loop in the MVP stage. Therefore, they will not be prioritized for development in this phase.

4.13 Product Manager's Judgment Summary

Although the product details page is just a front-end page, it connects product information, SKUs, inventory, shopping cart, orders, and the payment process. Therefore, in the MVP phase, I will prioritize ensuring three key aspects:

1. Product information should be clearly presented. Users need to quickly know what the product is, how much it costs, and whether it is in stock.
2. SKU and inventory are accurately linked. When a user selects a different SKU, the price and inventory must change in sync.
3. The purchase entry status must be clearly displayed. Information such as product being unavailable, sold out, no SKU selected, or insufficient stock must be clearly displayed, and users must be prevented from submitting invalid orders.

The core of this feature is not to make the page complicated, but to enable users to make purchasing decisions based on accurate information and smoothly enter the subsequent transaction process.

V. API Understanding Answers

This section analyzes the product details page API response provided by the engineering team, focusing on what information the interface can support the front end to display, and how the page should handle scenarios such as product status, price, inventory, and SKU selection.

5.1 Based on this API response, what information can be displayed on the product details page?

Based on the API response, the product details page can display the following information:

Information type	Corresponding fields	Page Display / Product Meaning
Product ID	productId	Used for system identification, API calls, order association, and event tracking statistics; it may not be directly displayed on the front end.
Product Name	productName	Displayed as product titles, such as Bluetooth Earphones
Product status	status	To determine whether a product is available for purchase, for example, ON_SALE indicates that it is currently on sale.

Selling price	price	Display current sales price
Original price	originalPrice	It can be displayed as a crossed-out price for price comparison.
Total inventory	stock	It displays the overall inventory of the product and can also be used to determine whether it is sold out.
Sales	salesCount	Displaying the product as "132 items sold" enhances user trust.
Product Images	images	Display product main image or image list
SKU Information	skuList	Displays specification options, such as White and Black, and is used to dynamically update prices and inventory.

From a product manager's perspective, this interface already supports basic product detail page display, including product name, image, price, inventory, sales volume, product status, and SKU selection. For the MVP stage, these fields are generally sufficient for users to understand the product and make a purchase decision.

5.2 How should the price of 19900 be displayed to the user?

`price = 19900` It should not be displayed as "19900". In e-commerce systems, price fields are usually returned in "cents" rather than directly in "yuan".

Therefore, the front end needs to `19900` Divide by 100 and display to the user: `19900 / 100 = 199.00`

Therefore, the page should display as follows: **¥199.00**

Similarly, `originalPrice = 29900` It should be displayed as: **¥299.00**

The page display can be designed as

Field	API return value	Page display
price	19900	¥199.00

originalPrice	29900	¥299.00
---------------	-------	---------

Black SKU price	20900	¥209.00
-----------------	-------	---------

Product managers need to pay special attention to pricing unit issues, as incorrect price display can directly impact user purchasing decisions and transaction trust. Displaying ¥199.00 as ¥19900 is not a sophisticated pricing strategy; it's a public blunder.

5.3 If status = OFF_SALE, how should the page handle this?

if **status = OFF_SALE** This indicates that the product has been removed from the shelves and users should not be allowed to continue purchasing it.

The page processing method is as follows:

Processing Items	Design Method
Product Information	You can continue to display basic information such as product name, image, and price.
Status prompts	The message clearly states, "This product is no longer available and cannot be purchased at this time."
Add to cart button	Gray out and disable
Buy Now button	Gray out and disable
Order Submission	Access to the order confirmation page is not allowed.
Guiding method	You can provide an option to "Return to product list" or "View similar products".

The core principles are: products can be displayed, but cannot be purchased; the status must be clear, and operations must be restricted.

This can prevent users from submitting invalid orders and also reduce the pressure on the backend for exception verification.

5.4 How should the "Buy" button be displayed if the total inventory of the product is 0?

If stock = 0, it means the product is currently out of stock. In this case, purchase-related operations should be disabled on the page.

Page elements	Handling method
Inventory display	exhibit "There is currently no stock of this product." or Sold out
Add to cart button	Gray out and disable
Buy Now button	Gray out and disable
Quantity selector	Disable or restrict users from increasing the number.
Button copy	Can be changed to Sold out or "Insufficient inventory"

It should be noted that even if the product status is still...ON_SALE If the inventory is zero, users should not be allowed to submit orders. Product status indicates whether a product is available for sale, while inventory determines whether a product is actually currently available for purchase. "Arrival notifications" can be omitted in the MVP stage because they do not affect the basic transaction loop and can be implemented as a feature in subsequent iterations.

5.5 If a user selects the Black SKU, how should the page price and inventory change?

If the user selects the Black SKU, the page should display accordingly. skuList The SKU information for Black is dynamically updated with prices and inventory.

The information for the Black SKU is as follows:

skuId = 20002

skuName = "Black"

price = 20900

stock = 15

Therefore, when a user selects the Black SKU, the price displayed on the page should be updated to:¥209.00

The inventory should be updated to:15 items in stock, This quantity is also the maximum purchase limit.

At the same time, the corresponding Black SKU should also be passed when submitting the order.skuld = 20002Instead of just passing the product ID, it should record the product ID. This is because different SKUs of the same product may have different prices, stock levels, colors, or specifications. If the order only records the product ID, it's impossible to accurately determine whether the user purchased White or Black, and it's also impossible to correctly deduct the corresponding SKU's stock.

Therefore, the page interaction logic after SKU selection can be described as follows:

The user selects the Black SKU → the page price is updated to ¥209.00 → the page inventory is updated to 15 pieces → when submitting the order, the skuld = 20002 is passed → the backend verifies the price and inventory based on this SKU.

If a certain SKU has 0 units in stock, it should be displayed as unavailable for purchase, and the user should not be able to select that specification to submit an order.

From a product manager's perspective, orders cannot only record the productId; the SKUId must also be recorded. This is because different SKUs for the same product may correspond to different prices, inventory levels, and shipping specifications. If only the product ID is recorded, it's impossible to accurately determine whether a user purchased White or Black, and it's also impossible to correctly deduct the corresponding inventory.

5.6 What fields are missing from this API response?

The current API already supports basic product detail pages, but to allow users to make a more complete purchase decision, it is recommended to add the following fields:

It is recommended to add fields.	effect	reason
productDescription	Product Details Description	The current interface only contains product names and images, lacking product selling points, parameters, and usage instructions.

categoryId / categoryName	Product Categories	Supports categorized display, filtering, searching, and operation management.
skuAttributes	SKU Attributes	The current skuName only displays White / Black, making it unclear whether this refers to color, size, or version.
shippingFee / deliveryInfo	Shipping and delivery information	Shipping costs and estimated delivery time can influence whether a user places an order.
returnPolicy / refundPolicy	After-sales policy	Before making a purchase, users usually pay attention to whether refunds, returns, or warranties are supported.
saleStartTime / saleEndTime	Shelf life or sales time	Supports scenarios such as scheduled product launches, limited-time sales, and pre-sales.
statusText	Status display text	Will ON_SALE / OFF_SALE Convert to user-understandable "Available for Sale / Discontinued"

Among them, the most recommended items to prioritize in the MVP stage are:

1.productDescription; 2.skuAttributes; 3.deliveryInfo; 4.returnPolicy; 5.statusText.

These fields can help product detail pages evolve from simply "displaying basic product information" to "supporting users' complete purchase decisions."

I believe the key to understanding this API is not memorizing field names, but rather understanding the business meaning behind the fields.

1. The price field requires attention to unit conversion to avoid incorrect amount display.
2. The status of a product directly affects whether a user can make a purchase.
3. SKU is the core unit for price and inventory judgment, and we cannot rely solely on total product inventory.
4. The information displayed on the front end can only be used as a reference for users. When submitting an order, the back end still needs to re-verify the product status, SKU, price, and inventory.
5. The interface fields must not only meet the requirements of page display, but also support user decision-making, order creation, inventory deduction, and subsequent data analysis.

Therefore, this API can already support the basic product detail page in the MVP stage, but it still needs to be supplemented with fields such as product description, SKU attributes, shipping and after-sales service to improve the completeness of users' purchase decisions.

VI. Code Logic Reading Answers

This section mainly analyzes the pseudocode given in the question, explains the meaning of the function in terms of business, and supplements the additional validation rules that need to be considered from the perspective of a product manager.

The pseudocode in the problem is as follows:

```
function canSubmitOrder(productStatus, skuStock, buyCount) {  
  if (productStatus !== 'ON_SALE') {  
    return {  
      canSubmit: false,  
      reason: 'Product is off sale'  
    };  
  }  
  if (skuStock <= 0) {  
    return {  
      canSubmit: false,  
      reason: 'Insufficient inventory'  
    };  
  }  
  if (buyCount > skuStock) {  
    return {  
      canSubmit: false,  
      reason: 'Purchase quantity exceeds available inventory'  
    };  
  }  
  return {  
    canSubmit: true,  
    reason: ''  
  };  
}
```

}

6.1 What is this function primarily used to determine?

This function is mainly used to determine whether the current product and SKU meet the order placement conditions before the user submits an order.

It makes judgments based on three core conditions:

1. Is the product currently available for sale?
2. Is the current SKU in stock?
3. Does the number of items purchased by the user exceed the SKU inventory?

Only when the product status is `ON_SALE` The function will only return if the SKU inventory is greater than 0 and the purchase quantity does not exceed the inventory. `canSubmit: true` This indicates that a user can submit an order.

From a product manager's perspective, this code can be understood as the basic business validation rules before order submission. Its value lies not in the complexity of the code itself, but in how it transforms product status, inventory, and purchase quantity into clear order restrictions.

6.2 Under what circumstances can a user not submit an order?

According to the function logic, users cannot submit orders in the following three situations.

Judgment conditions	Return results	Business meaning	Page prompts and suggestions
<code>productStatus !== 'ON_SALE'</code>	<code>canSubmit: false</code>	The product is not currently available for sale; it may have been removed from the shelves or is unavailable for purchase.	The product is no longer available and cannot be purchased.

<code>skuStock <= 0</code>	<code>canSubmit: false</code>	There is currently no available stock for this SKU.	Current specifications are in short supply.
<code>buyCount > skuStock</code>	<code>canSubmit: false</code>	The number of items purchased by the user exceeds the current SKU inventory.	The quantity purchased exceeds available stock. Please adjust the quantity.

It should be noted that the goods are in **ON_SALE**. This doesn't guarantee a purchase. Users must also select a valid SKU, and that SKU must have sufficient stock. In e-commerce systems, "product in stock" and "the selected specification in stock" are not the same thing; many of the frustrations in the human shopping experience stem from these seemingly minor details.

6.3 If `productStatus = 'ON_SALE'`, `skuStock = 3`, and `buyCount = 5`, what result should the function return?

In this case:

`productStatus = 'ON_SALE'`

`skuStock = 3`

`buyCount = 5`

First, the product status is **ON_SALE**. This indicates that the product is available for sale and therefore passes the first layer of verification.

Secondly `skuStock = 3`. Since the inventory is greater than 0, it can pass the second-level verification.

but, `buyCount = 5`. A user wants to buy 5 items, but the current stock of SKU is only 3. Since the purchase quantity exceeds the stock, the following applies: `buyCount > skuStock`

Therefore, the function will enter the third condition and return the result that the order cannot be submitted.

The function should return:

```
{
  canSubmit: false,
  reason: 'Purchase quantity exceeds available inventory'
}
```

}

The corresponding page prompt is "[Only 3 items remain in stock. Please modify the quantity before submitting your order.](#)"

This scenario illustrates that even if a product is available for sale and the SKU is in stock, the quantity a user can purchase is still subject to the inventory limit.

6.4 What additional checks should be added?

The current function only covers three basic conditions: product status, SKU inventory, and purchase quantity. For a real e-commerce system, the following validations should be added before submitting an order:

Verification Items	Verification content	Business reasons	Page prompts and suggestions
User login status	Determine if the user is logged in	Orders need to be linked to a user ID for easy payment, after-sales service, and order tracking.	Please log in before submitting your order.
Shipping address	Check if the address is complete and valid	Physical goods must have a shipping address to be shipped.	Please fill in a valid shipping address.
Legality of purchase quantity	buyCount Must be a positive integer	Prevent 0, negative numbers, decimals or illegal characters	Please enter a valid purchase quantity.
Does the product exist?	Determine if productId is valid	The product may have been removed or expired.	The product is unavailable or expired.
Is the SKU valid?	Determine if the skuId exists and belongs to the current product.	Avoid binding incorrect specifications to orders	Product specifications are invalid, please select again.

Price verification	The backend re-validates the current price of the product or SKU.	To prevent front-end prices from expiring or being tampered with	The product price has been updated, please confirm again.
Purchase restriction rules	Determine if the purchase limit for a single person has been exceeded.	Suitable for promotional, scarce, or subsidized products	The quantity of this item has exceeded the purchase limit.
Payment methods	Determine if the current payment method is available	Prevent users from choosing unavailable payment channels	The current payment method is unavailable.
Shopping cart item status	Check if the items in your shopping cart are unavailable, expired, or have changed price.	To prevent users from submitting invalid items from their old shopping carts	There are invalid items in your shopping cart. Please modify them and submit again.
Risk control verification	Determine if the account, order amount, or order placement behavior is abnormal.	Prevent fraudulent orders, malicious purchases, or abnormal transactions.	The order has an error. Please try again later or contact customer service.

From an MVP perspective, not all validations need to be overly complex in the first version, but login status, shipping address, purchase quantity validity, SKU validity, price validation, and inventory validation should be prioritized. These directly impact whether orders can be generated correctly.

6.5 Summary

This pseudocode illustrates a very basic but crucial order submission validation logic in an e-commerce system. The front-end can display product information and guide the user's actions, but whether to allow order submission ultimately depends on the back-end's determination of product status, SKU inventory, and purchase quantity.

From a product manager's perspective, I would focus on three key points:

1. The conditional statements in the code need to be translated into business prompts that users can understand. For example, "Product is off sale" should not be directly displayed

to the user, but should be translated into..."The product is no longer available for purchase."。

2. Orders must be verified by the backend before submission. This is because the inventory and prices displayed on the frontend page may be outdated, or may be inaccurate due to caching, concurrent purchases, or changes in product status.
3. Basic validation should prioritize covering key conditions affecting order generation, such as product status, SKU, inventory, purchase quantity, user login, shipping address, and price. More complex risk control, purchase limits, and payment method rules can be gradually expanded as needed based on business phases.

Therefore, although the code is simple, it reflects the ability that product managers need to have: to understand the business rules behind the code and to translate them into clear product logic, page prompts, and acceptance criteria.

VII. Order Status Understanding Questions

Answers

This section primarily analyzes the order status design in e-commerce systems. Order status describes the current business stage of an order and determines what actions users, operations personnel, and the system can perform at that stage.

The order status given in the question includes:

- **PENDING_PAYMENT**Pending payment
- **PAID**Paid
- **SHIPPED**Shipped
- **COMPLETED**Completed
- **CANCELLED**Cancelled
- **REFUNDING**Refund in progress
- **REFUNDED**Refund has been issued.

I believe the core of order status design is not simply listing the statuses, but ensuring clear status transitions, well-defined triggering conditions, consistent front-end and back-end display, and proper handling of abnormal scenarios.

7.1 What should be the initial status after a customer submits an order?

After a customer submits an order, the initial status of the order should be: **PENDING_PAYMENT** Pending payment

The reason is that the user clicked [Submit an order](#). This only indicates that the order has been successfully created, but payment has not yet been completed. At this point, the transaction is not truly established, and the system cannot directly mark the order as paid or completed.

In this state, users can typically perform the following operations:

1. Continue making payments;
2. Cancel the order;
3. Return to the order list;
4. View order details.

If the user fails to complete the payment within the payment deadline, the system should automatically cancel the order and release the corresponding inventory.

7.2 What should the order status be after successful payment?

After successful payment, the order status should change from: **PENDING_PAYMENT** Pending payment → **PAID** Paid

PAID This indicates that the system has confirmed the user has completed payment, and the order has entered the pending shipment stage. At this point, operations staff can view the order in the backend and arrange subsequent shipment.

After successful payment, the system should record the following information:

1. Payment time;
2. Payment transaction number;
3. Payment amount;
4. Payment methods;
5. Payment result.

It's important to note that order status updates should be based on the backend receiving a successful payment callback from the payment platform or actively querying for confirmation,

not solely on the frontend payment page display. Otherwise, if the user's page shows success, but the backend hasn't received the payment, and the merchant has already shipped the goods, that's not e-commerce; it's charity, or at least the kind where the system automatically participates.

7.3 What should the order status be after the merchant ships the goods?

After the merchant ships the goods, the order status should change from: **PAID** Paid → **SHIPPED** Shipped

SHIPPED This indicates that the merchant has processed the order and shipped it, and the order has entered the logistics and delivery stage.

State changes can be described as:

Order payment successful → Status: **PAID**

The operations staff fills in the logistics information and confirms the shipment → Status updated to **SHIPPED**

exist **SHIPPED** In this state, the system typically needs to record the following information:

1. Delivery time;
2. Logistics company;
3. Tracking number;
4. Delivery status;
5. Delivery address;
6. Estimated delivery time;
7. Shipping operator.

The client page can display: "The merchant has shipped the goods, and they are being delivered." It also provides a logistics tracking portal, for example: "check logistics"

The operations backend should then remove the order from the "Pending Shipment" list and move it to the "Shipped" or "Delivering" status.

7.4 What should the order status be if the customer fails to make payment within the payment deadline?

If the customer fails to complete payment within the payment deadline, the order status should change from: **PENDING_PAYMENT** Pending payment

→ **CANCELLED** Cancelled

The reason is that orders have been created but not paid for. If they are held for a long time, they will occupy inventory and affect the number of goods available for sale. Therefore, the system needs to automatically cancel the orders through scheduled tasks or order timeout mechanisms.

When payment is cancelled due to timeout, the system should perform the following operations:

1. Update the order status to **CANCELLED**;
2. Release orders to lock in inventory;
3. Record the reason for cancellation, for example **Payment Timeout**;
4. The order details page will display that the order has been cancelled.
5. If the user still wants to purchase, guide them to place a new order.

and **CANCELLED** Cancellations can be triggered by various reasons, such as user-initiated cancellation, payment timeout cancellation, backend cancellation, or risk control cancellation.

Therefore, in addition to the order status, the system should also record the cancellation reason to facilitate subsequent analysis of order loss issues.

7.5 Under what circumstances should a customer be allowed to request a refund?

Whether a customer can apply for a refund depends on the order status and business rules.

Order status	Is a refund allowed?	Judgment Explanation
PENDING_PAYMENT	Not allowed	The user has not yet paid, so there is no refund issue, and the order can be cancelled.
PAID	allow	For payments made but goods not shipped, the refund process is relatively simple.

SHIPPED	Allowed	Whether refunds are supported after shipment depends on the return and after-sales policy.
COMPLETED	Allowed	If it's still within the warranty period, a refund is supported.
CANCELLED	Not allowed	Orders have been cancelled, and refunds are usually not requested.
REFUNDING	Duplicate applications are not allowed	The order is currently being processed for a refund.
REFUNDED	Not allowed	The refund has been completed and should not be requested again.

A more reasonable basic design is:

1. **PAID** Refunds are allowed under certain conditions. Since the goods have not yet been shipped, the refund process is relatively simple;
2. **SHIPPED** Refunds are allowed under the current status. However, this usually requires consideration of logistics or merchant verification regarding refused delivery and returns;
3. **After-sales** refunds are allowed when the product is **COMPLETED**. However, after-sales time limits and product rules must be met;
4. **The following types of refunds are not allowed to be requested repeatedly: PENDING_PAYMENT, CANCELLED, REFUNDING, and REFUNDED.**

The MVP stage can initially support the basic refund process in the PAID state, while the complex after-sales processes in the SHIPPED and COMPLETED states can be simplified or used as subsequent iterations.

7.6 Order Status Transition Process Description

The basic order status flow can be divided into three categories: normal transaction process, cancellation process, and refund process.

1. Normal transaction process

Under normal circumstances, orders will flow through the following path:

PENDING_PAYMENT → PAID → SHIPPED → COMPLETED

The meaning is as follows:

State transition	Triggering event
PENDING_PAYMENT → PAID	User payment successful
PAID → SHIPPED	Merchant confirms shipment
SHIPPED → COMPLETED	The user confirms receipt or the system automatically confirms receipt.

This process is the core transaction loop of an e-commerce system.

2. Payment timeout or user cancellation process

If a user submits an order but does not complete payment, the order can be withdrawn from [the payment process]. **PENDING_PAYMENT** The state becomes **CANCELLED**

The state transition is as follows: **PENDING_PAYMENT → CANCELLED**

Triggering scenarios include:

- The user canceled the order.
- The user has not made payment within the payment deadline;
- The system detected an order anomaly and cancelled it.
- The operations staff canceled the order in the backend.

In this process, if an order has not yet been paid, the inventory should be released upon cancellation, but a refund is not required.

3. Refund Process

If the user has already paid for the order, the refund process can be initiated based on the order status.

The typical refund status progression is as follows: **PAID → REFUNDING → REFUNDED**

This applies to orders that have been paid for but not yet shipped.

If the order has already been shipped, it may also enter the refund process: **SHIPPED → REFUNDING → REFUNDED**

This applies to scenarios where users apply for returns and refunds, refuse to accept goods, or the merchant agrees to a refund.

If the order has been completed but is still within the after-sales period, it may also enter the refund process: **COMPLETED** → **REFUNDING** → **REFUNDED**

This applies to scenarios where the platform supports after-sales refunds.

in: **REFUNDING** This indicates that the refund request has been submitted and is awaiting review or processing.

REFUNDED This indicates that the refund has been completed and the order amount has been returned to the user's account.

7.7 Order Status Transition Diagram

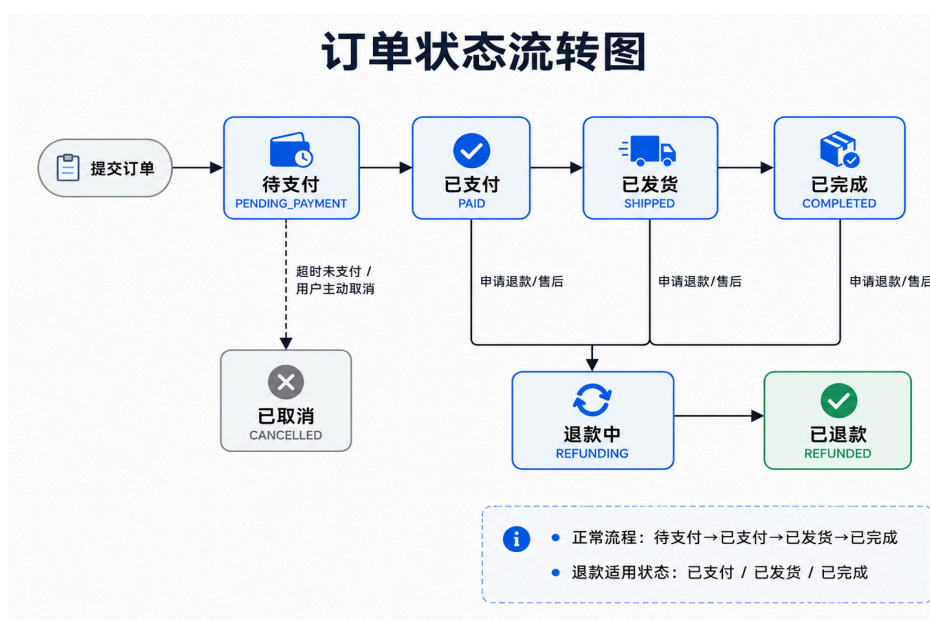


Figure 7-7 Order Status Transition Diagram

7.9 Summary

There are four key issues to consider when designing order status.

1. States must be mutually exclusive. An order can only be in one primary state at a time; it cannot be in multiple states simultaneously. **PAID** and **CANCELLED**.
2. State changes must be triggered by a specific event. For example, a successful payment triggers this. **PENDING_PAYMENT** arrive **PAID** Merchant shipment trigger **PAID** arrive **SHIPPED**.
3. Key status changes need to be recorded in terms of time and reason. For example, payment time, shipping time, cancellation time, refund application time, cancellation

reason, and refund reason. This information is important for user inquiries, operational processing, and data analysis.

4. The front-end display status and the actual back-end status must be consistent. If the statuses of the user client, the operations back-end, and the payment system are not synchronized, it will directly lead to user complaints and order processing chaos.

Therefore, the goal of order status design is not to have as many statuses as possible, but to make the transaction process clear, traceable, and recoverable, and to support core scenarios such as normal transactions, cancellations, and refunds.

VIII. Bug/Exception Analysis Answers

This section primarily analyzes the inventory anomaly issues reported by the QA team:

Users see that the product inventory is 1 on the product details page, but when submitting an order, the system displays the message "Insufficient inventory".

On the surface, this appears to be an issue where the inventory displayed on the product details page differs from the inventory verified by the backend when an order is submitted. However, in a real e-commerce system, this problem could be caused by a variety of factors, including caching, concurrent purchases, inconsistent SKU inventory, changes in the status of items in the shopping cart, data asynchrony between the front-end and back-end, and the order submission interface re-verifying the inventory.

From a product manager's perspective, troubleshooting this issue should not be limited to determining whether "the front-end is wrong" or "the back-end is wrong." Instead, it should involve a step-by-step analysis that considers user operation paths, API responses, SKU inventory, order records, inventory flow, and caching mechanisms.

The core judgment principle is: **The inventory displayed on the front end is only for user reference; the final decision is based on the real-time verification result from the backend when an order is submitted.**

8.1 Possible Cause Analysis

Possible reasons	Detailed Explanation	Determine direction
The product details page has a cached inventory.	The inventory seen by the user is cached data, but the database inventory has already become 0.	Check if the product details page API reads from the cache, and whether the cache is refreshed promptly after inventory changes.
Other users placed orders first.	After a user sees that the inventory is 1, another user submits an order or successfully pays, thus occupying the last remaining inventory.	Check if any other orders deducted or locked the inventory of this SKU during the problematic period.
Total product inventory does not match SKU inventory.	The page displays a total product inventory of 1, but the inventory of the specific SKU selected by the user is 0.	Check whether the page displays product stock or SKU stock.
Users submit old items from shopping carts	The item was in stock when added to the cart, but the stock had changed or the item was expired when the order was submitted.	Check if the shopping cart page re-verifies the product status, SKU, and inventory.
The front end has not refreshed the inventory.	Users remain on the product details page for an extended period, even though inventory has changed but the page has not updated.	Reproduce the inventory changes after the user stays on the page.
Order submission API re-verify inventory	The order placement interface uses real-time inventory verification and detects insufficient stock.	Check the order submission interface logs and backend inventory verification logic.

The rules for locking or deducting inventory are unclear.	Inventory may be locked by unpaid orders, resulting in insufficient available stock.	Check inventory lock records and order status
Front-end parameter passing error	Error when submitting orders or <code>buyCount</code> This caused the backend verification to fail.	Check the order submission request parameters

8.3 Explanation of Key Reasons

8.3.1 Inventory data is cached on the product details page.

Product detail pages typically see high traffic, and to improve page load speed, the system may cache product information. If the inventory field is also cached, a situation may arise where the user sees an inventory of 1, but the real-time inventory in the database is already 0.

During the investigation, the following needs to be confirmed:

1. Does the product details API read from the cache?
2. Should the inventory field be cached along with basic information such as product name and image?
3. Does the cache expire or update promptly after inventory changes?
4. When the problem occurs, are the database inventory and cache inventory consistent?

The optimization suggestion is: low-frequency changing information such as product name, image, and description can be cached, but the caching time for high real-time fields such as inventory should be shortened as much as possible, or the real-time inventory should be retrieved again before the order is submitted.

8.3.2 Other users preemptively purchase the last remaining stock

If there is only one item left in stock, it is easy for multiple users to compete for the same stock when they buy at the same time.

The possible process is as follows:

1. User A opens the product details page and sees that the inventory is 1;
2. User B completes the order or payment before User A submits the order;
3. The system locks or deducts the last item from the inventory.
4. When User A submits an order, the backend detects insufficient inventory.
5. System return “Insufficient inventory”。

This situation isn't necessarily a system bug; it could be a normal concurrency scenario.

E-commerce systems don't reserve inventory for users just because they glance at a product; otherwise, the inventory would be silently taken over by a bunch of hesitant humans.

The following need to be checked during the investigation:

1. The time when the user sees the inventory as 1;
2. The time when the user submits the order;
3. Were there any other orders occupying this SKU during this period?
4. Does the inventory deduction or locking occur at the order submission stage or the payment success stage?

8.3.3 Inconsistency between total product inventory and SKU inventory

The product details page may display the total stock of the product, but the specific SKU stock is verified when the order is submitted.

For example:

Product / SKU in stock

Total Inventory 1

White SKU 1

Black SKU 0

If the page displays "Inventory 1" but the user selects the Black SKU, the backend will return an insufficient inventory when verifying the Black SKU inventory.

During the investigation, the following needs to be confirmed:

1. Does the page display total inventory or current SKU inventory?
2. After a user selects an SKU, does the page dynamically update the inventory?
3. Did you pass the correct skuld when submitting the order?
4. Does the backend verify inventory based on skuld?
5. Does the total inventory of goods equal the sum of the inventory of all SKUs?

From a product design perspective, after a user selects an SKU, the page should display the current SKU's inventory, rather than continuing to display the total inventory of the product.

8.3.4 The status of items in the shopping cart is not updated in a timely manner.

If a user submits an order from their shopping cart, inventory changes may occur after the item has been added to the cart.

For example:

- The inventory was 1 when the user added the item to their cart.
- The product was subsequently purchased by other users or removed from the platform by the platform's management.
- The shopping cart still displays information about old items;
- Users submit orders directly;
- The backend detected insufficient inventory during re-verification.

In this case, the key issue is whether the shopping cart page re-verifies the product status before the user enters, selects items, modifies quantities, and submits the order.

The following aspects need to be carefully verified in the shopping cart:

1. Is the product still valid?**ON_SALE**;
2. Are the SKUs still valid?
3. Is the SKU inventory sufficient?
4. Has the price of the goods changed?
5. Has the product been deleted or removed from the platform?

8.4 Recommended troubleshooting steps

step	Investigation content	Purpose
1	Confirm user operation path	Determine whether the user purchases immediately from the product details page or submits an order from the shopping cart.
2	View user-submitted parameters	Please verify that the productId, skuId, buyCount, userId, and submission time are correct.

3	Check the return value of the product details interface	Determine whether the inventory returned by the API is the total inventory of products or the inventory of SKUs.
4	Check the front-end display logic	Confirm whether the page dynamically updates inventory based on the SKU selected by the user.
5	Check order submission API logs	View the specific conditions for determining insufficient inventory in the backend.
6	Check inventory flow	Confirm the time, reason, and triggering order for the inventory changing from 1 to 0.
7	Check order records	Determine if any other orders were using the last available inventory before the user submitted their order.
8	Check caching mechanism	Determine if the cached inventory is inconsistent with the real-time inventory in the database.
9	Determine the problem type	Distinguish between normal concurrency issues, front-end display problems, caching issues, SKU issues, and back-end validation issues.

When troubleshooting these types of issues, inventory records are crucial. If the system doesn't record the reasons for inventory changes and the corresponding orders, locating the problem becomes extremely difficult.

8.8 Product-level optimization suggestions

Optimization direction	Specific suggestions	Product Value
Recheck inventory before submitting order	Before the user clicks "submit order," the backend re-verifies the product status, SKU, price, and inventory.	Avoid relying on expired front-end data
SKU inventory display is prioritized.	After a user selects an SKU, the page displays the current SKU inventory, not the total inventory of the product.	Reduce user misunderstandings

Low inventory alert	When inventory is 5 or less, display "Limited stock, please place your order as soon as possible".	Reduce users' misunderstandings about inventory fixedness
Optimize error messages	Translate "Insufficient inventory" into user-understandable copy	Improve user experience
Shopping cart real-time verification	Before users add items to their shopping cart, select products, modify quantities, and submit orders, the system will re-verify the transaction.	Prematurely exposing product failures or insufficient inventory
Cache strategy optimization	Basic product information can be cached, and inventory fields can have their caching time shortened or be retrieved in real time.	Reduce inconsistencies in inventory display
Inventory transaction records	Record the time, order, and reason for inventory deduction, locking, and release.	Convenient for troubleshooting and data tracking
Differentiating the causes of failure	The backend returns more specific error codes, such as SKU_OUT_OF_STOCK and PRODUCT_OFF_SALE.	To facilitate accurate display of prompts on the front end

The error message suggests changing the system language to the user language:

System return	User Showcase Suggestions
Insufficient inventory	Product inventory has changed. Please select a different product or modify the quantity you wish to purchase.
SKU_OUT_OF_STOCK	The current specification is sold out. Please select another specification.
PRODUCT_OFF_SALE	This product has been removed from the shelves and is temporarily unavailable for purchase.

PRICE_CHANGED

The product price has been updated. Please confirm the price before placing your order.

8.9 Summary

The essence of this problem is the inconsistency between the data displayed on the front end and the real-time validation results on the back end. For e-commerce systems, this type of issue cannot be completely avoided, especially in scenarios with low inventory and multiple users making concurrent purchases. However, product managers need to mitigate the impact of this problem on user experience through rule design, page prompts, and troubleshooting mechanisms.

My judgment is:

1. The order submission interface must be the final verification source. Front-end inventory can only be used for display and reference, and cannot be used as the basis for placing an order.
2. Product detail pages and shopping carts should display inventory changes as early as possible. Don't wait until users click to submit their order to discover that a product is unavailable.
3. SKU inventory should be displayed before total product inventory. Users are purchasing specific SKUs, not abstract products.
4. The system needs to record inventory flow and failure reasons. Otherwise, subsequent troubleshooting will rely solely on guesswork, which is the cheapest and most dangerous form of entertainment in product management.

Therefore, resolving this issue requires not only technical troubleshooting but also product-side optimization of user prompts, inventory display logic, shopping cart validation, and error reporting. The ultimate goal is to help users understand why they cannot place an order, while enabling product and development teams to quickly pinpoint the source of the problem.

IX. Data Metrics Design

After the e-commerce system's MVP is launched, the product manager needs to use data to determine whether the core transaction process is truly functioning correctly. For the first version

of the system, the goal of the data metrics is not to build a complex data analysis system, but rather to answer a few key questions:

1. Whether users can see the products;
2. Whether the user is interested in purchasing the product;
3. Whether users can successfully submit orders;
4. Whether the user is able to successfully complete the payment;
5. Whether the merchant can complete the delivery and fulfillment;
6. Are users lost due to cancellations, refunds, or inventory issues?

Therefore, the data metrics in this solution are designed around the following core links:

Browse products → Add to cart / Buy now → Submit order → Payment successful → Ship → Order complete

From a product manager's perspective, data metrics in the MVP phase should serve to "verify the transaction loop" and "identify issues along the chain." If, after the system goes live, you only look at GMV without considering add-to-cart rate, payment success rate, order completion rate, and order cancellation rate, it's difficult to pinpoint where the problem occurred. Focusing only on the overall result without considering the process is like looking only at exam scores without examining which questions were answered incorrectly—it yields nothing but anxiety.

9.1 Core Indicator Table

During this MVP phase, it is recommended to focus on monitoring the following 8 core metrics:

Indicator Name	Indicator Meaning	Calculation method	importance
Product detail page views	Number of times or number of users viewed the product details page	Product details page PV / UV	Measuring product exposure and user browsing interest is the starting point of the conversion funnel.
Add-to-cart rate	Percentage of users who added products to their cart after browsing them	Number of users adding items to cart / Number of users visiting the product details page	Measuring product appeal and user purchase intention

Order submission conversion rate	Percentage of users who successfully submitted an order after browsing the products	Number of users submitting orders / Number of users visiting the product details page	Measure the conversion rate from user browsing to placing an order.
Payment success rate	The percentage of orders that successfully completed payment after submission.	Number of successfully paid orders / Number of orders submitted	Measuring the stability and smoothness of the payment process
GMV	Total transaction amount of paid orders	Total amount of paid orders	Measuring the overall transaction volume of the platform
Order completion rate	The percentage of completed orders out of paid orders	Number of completed orders / Number of paid orders	Measuring order fulfillment and transaction completion status
Cancellation rate	The percentage of cancelled orders out of submitted orders	Number of cancelled orders / Number of submitted orders	Help identify issues such as payment timeouts, inventory changes, and price hesitation.
Refund rate	The percentage of refunded orders out of paid orders	Number of refunded orders / Number of paid orders	Measuring product quality, after-sales experience, and user satisfaction

These eight metrics cover the main path from product exposure to transaction completion, allowing you to observe user conversion as well as payment, fulfillment, and after-sales issues. For the MVP stage, the metrics don't need to be overly complex from the start, but they must help the team determine whether the system is usable, whether the process is smooth, and where the problems are concentrated.

9.2 Hierarchical Design of Indicators

To avoid simply listing the metrics, the above metrics can be divided into three categories: user conversion metrics, transaction result metrics, and fulfillment and after-sales metrics.

9.3.1 User Conversion Metrics

User conversion metrics are primarily used to determine whether a user is willing to move from browsing to the purchase process.

index	Focus on issues	Possible issues
Product detail page views	Did the user see the product?	Insufficient product exposure, unreasonable list sorting, and weak search entry point
Add-to-cart rate	Does the user have a purchase interest?	Product price, images, descriptions, SKUs, and inventory display influence purchase intentions.
Order submission conversion rate	Did the user successfully place the order?	The shopping cart process is complicated, address entry is difficult, inventory is insufficient, and the checkout page has a poor user experience.

If a product detail page has high page views but a low add-to-cart rate, it means that although users have entered the product detail page, the product information, price, or page experience has not effectively motivated them to make a purchase. If the add-to-cart rate is high but the order submission conversion rate is low, then you need to focus on checking the shopping cart, SKU validation, address entry, and order confirmation page.

9.3.2 Transaction Result Indicators

Transaction outcome metrics are primarily used to determine whether a user has actually completed the payment and whether the platform has generated transaction value.

index	Focus on issues	Possible issues
Payment success rate	Has the user successfully paid after submitting the order?	Payment channel error, payment page redirection failure, payment timeout, user abandonment
GMV	Platform actual transaction amount	Changes in sales volume, product pricing, order quantity, and average order value

Payment success rate is a crucial metric in the MVP (Most Valuable Product) phase. Submitting an order does not equate to a completed transaction; only after successful payment does the order proceed to the subsequent shipping and fulfillment stages. A low payment success rate indicates a potential problem in the payment process, requiring priority investigation. GMV reflects the platform's transaction volume but cannot solely determine business health. For example, an increase in GMV coupled with a rising refund rate may indicate issues with product quality or user expectations. Therefore, GMV should be analyzed in conjunction with payment success rate, order completion rate, and refund rate.

9.3.3 Performance and After-sales Indicators

Fulfillment and after-sales metrics are used to determine whether an order has been truly completed and whether the user is satisfied after purchasing.

index	Focus on issues	Possible issues
Order completion rate	Has the paid order been finally completed?	Shipping efficiency, logistics anomalies, insufficient inventory, and back-end processing efficiency
Cancellation rate	Why was the user unable to complete the transaction?	Payment timeout, user hesitation, price change, insufficient inventory
Refund rate	Are users satisfied after their purchase?	Product quality issues, inaccurate descriptions, poor delivery experience, and unclear after-sales policies.

Order completion rate can determine whether the transaction loop is truly running smoothly. An e-commerce system doesn't end after the user pays; it also requires shipping, logistics, and confirmation of receipt. Cancellation rate and refund rate can help product managers pinpoint problems. A high cancellation rate may indicate issues before payment; a high refund rate often indicates problems after payment, such as product quality, logistics, or after-sales experience.

9.3 Indicator Funnel Analysis

In addition to monitoring each metric individually, it is also necessary to combine the metrics into a transaction conversion funnel to observe at which stage users mainly churn.

A basic e-commerce conversion funnel can be designed as follows: **View product details page** → **Add to cart / Buy now** → **Submit order** → **Payment successful** → **Order complete**

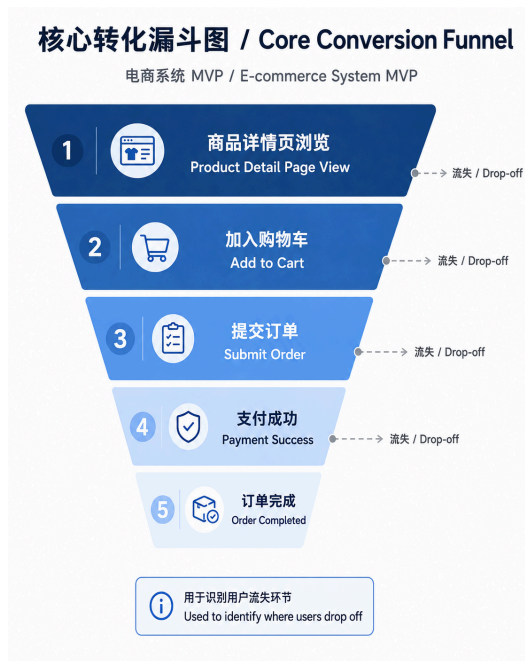


Figure 9-3: Core Conversion Funnel Diagram

By analyzing the funnel, the following judgments can be made:

Funnel stage	If the loss is severe, it may indicate...
Low product detail page views	Insufficient product exposure, weak entry points to product lists, and poor search or categorization experience.
High pageviews but low add-to-cart rate	Product images, prices, descriptions, SKUs, or inventory displays are not attractive enough.
High add-to-cart rate but low order submission rate	There are issues with the shopping cart, order confirmation page, address entry, or inventory verification.
High order submission rate but low payment success rate	There are issues with the synchronization of payment processes, payment channels, or payment results.

High payment success rate but low order completion rate Problems exist in shipping, logistics, inventory fulfillment, or back-end processing.

High refund rate after order completion There are problems with product quality, product description, user expectations, or after-sales experience.

Funnel analysis is more valuable than simply looking at GMV because it helps teams pinpoint specific problem areas. For example, a decline in GMV is just a result; the real problem might be decreased product exposure, lower add-to-cart rates, increased payment failures, or a higher refund rate. Product managers can't just stare at the results; that's data worship, not data analysis.

9.11 MVP Phase Tracking Recommendations

To support the above metrics, basic event tracking needs to be designed in advance during the MVP phase. The tracking doesn't need to be overly complex initially, but it must cover key user behaviors from browsing products to completing an order.

Event tracking	Triggering timing	use
product_detail_view	User enters product details page	Statistics on product detail page views and product exposure
sku_select	User selects SKU	Analyzing product specifications based on user preferences
add_to_cart_click	The user clicks "Add to Cart"	Analysis of purchase intentions
add_to_cart_success	Item successfully added to cart	Statistics on the success rate of adding items to cart
buy_now_click	The user clicked "Buy Now"	Analysis of direct purchase intentions
submit_order	User successfully submitted order.	Statistics on order submission conversion rate
payment_success	User payment successful	Statistics on payment success rate and GMV

payment_failed	User payment failed.	Analysis of payment failure reasons
cancel_order	User cancels order or system cancels order	Analyze the reasons for order cancellation
ship_order	Operations staff confirm shipment	Statistical performance of contracts
order_completed	The user confirms receipt or the system completes the process automatically.	Statistical order completion rate
request_refund	User requests refund	Analyze after-sales issues
refund_success	Refund completed	Statistical refund rate

It is recommended to record the following basic fields for each event:

Field	illustrate
userId	User ID, used to analyze user behavior paths
productId	Product ID, used to analyze product performance
skuId	SKU ID is used to analyze conversion and inventory status of different specifications.
orderId	Order ID, used to associate order status and payment status
eventTime	Time of the incident
pageSource	Page source, such as product list, search page, shopping cart
price	Current price of product or SKU
buyCount	Number of user purchases
orderStatus	Current order status

failureReason Reasons for payment failure, submission failure,
or cancellation

From a product manager's perspective, event tracking fields should not only serve to "count quantities," but also help the team troubleshoot problems. For example, without failureReason for payment failure events, all you know is that a payment failed, but you don't know why it failed. Looking at this kind of data will only prolong meetings and serve no purpose.

9.6 Product Actions Following Indicator Monitoring

The value of metric design lies not in simply "listing" them, but in guiding subsequent product optimization. After the MVP is launched, different product actions can be taken based on the performance of different metrics.

Data performance	Possible problems	Product optimization direction
Low product detail page views	Insufficient product exposure	Optimize the display of product lists, category entry points, search results, and product cards.
Low add-to-cart rate	Product lack of appeal	Optimize the main image, price display, product description, SKU display, and the position of the buy button.
Low order submission conversion rate	There are obstacles in the order placement process.	Simplify the shopping cart and order confirmation page, and optimize address entry and inventory display.
Low payment success rate	Unstable payment chain	Check payment channels, payment redirects, payment timeouts, and callback synchronization.
High order cancellation rate	User placed an order and then cancelled.	Analyze the reasons for cancellations and optimize payment deadlines, shipping cost display, and price confirmation.

Low order completion rate	There are problems with contract performance.	Optimize the backend shipping process, logistics information entry, and abnormal order alerts.
High refund rate	There are problems with the product or after-sales service.	Analyze the reasons for refunds and optimize product descriptions, quality control, and after-sales rules.
A certain SKU has a low conversion rate	Insufficient attractive specifications or abnormal inventory	Adjust SKU display order, replenish inventory, or optimize specification naming.

For example, if the payment success rate is significantly low, the product manager needs to prioritize investigating payment channels, payment page redirects, payment callbacks, and payment timeout logic, rather than blindly optimizing the product details page. If the refund rate is significantly high, the product description, images, after-sales policies, and user reviews should be reviewed to determine if there are discrepancies between user expectations and the actual product. Therefore, data metrics should be developed..."**Identify the problem → Locate the issue → Propose optimizations → Verify the results**"A closed loop.

9.12 Summary

The data metrics design in the MVP phase doesn't need to be overly complex from the start, but they must be able to answer key questions in the transaction chain:

1. Did the user see the product?
2. Whether the user has the intention to purchase;
3. Can users successfully submit their orders?
4. Will the user be able to successfully make the payment?
5. Can the order be fulfilled?
6. Are users lost due to cancellations or refunds?

This solution selects product detail page views, add-to-cart rate, order submission conversion rate, payment success rate, GMV, order completion rate, order cancellation rate, and refund rate as core metrics because they cover the entire path from product exposure to transaction completion.

From a product manager's perspective, data metrics aren't just for making reports look richer, but for helping the team determine if the system is truly functioning correctly and where problems occur. After the first version of the MVP goes live, the team should prioritize payment success rate, order completion rate, order cancellation rate, and refund rate, as these metrics directly reflect the stability of the core transaction process.

As the business grows, metrics such as average order value, repurchase rate, product conversion rate, SKU sales structure, channel conversion rate, and user retention rate can be further expanded. However, in the first stage, the most important thing is to first verify the basic transaction loop, and then gradually build a more complete data analysis system.

10. Summary and Future Iteration Directions

This solution is designed around the MVP version of a basic e-commerce system, with the core objective of prioritizing the completion of the basic transaction loop from product browsing to order completion.

Browse products → Add to cart / Buy now → Submit order → Pay → Ship → Order complete

In the user role design, this solution clearly defines three core roles: consumers, operations personnel, and platform administrators. Consumers focus on successfully completing product browsing, SKU selection, order placement and payment, and order inquiry; operations personnel focus on product management, inventory maintenance, order processing, and fulfillment; and platform administrators mainly undertake basic governance responsibilities such as access control, data viewing, and handling abnormal orders.

In the MVP feature planning, this solution prioritizes features according to P0, P1, and P2. P0 features mainly revolve around products, SKUs, inventory, orders, payments, and shipping, ensuring the system can complete the most basic transaction processes. P1 features are used to improve user experience and operational efficiency, such as search filtering, refund processing, data dashboards, and basic permission management. P2 features primarily serve subsequent growth and refined operations, such as coupons, membership systems, product reviews, and personalized recommendations.

In its core process design, this solution focuses on streamlining the user order placement and payment process, as well as the product creation and publishing process. The former determines whether a user can complete a purchase, while the latter determines whether the

platform has accurate and available products for the user to buy. These two processes together support the basic business loop of the e-commerce system. Simultaneously, this solution also covers abnormal scenarios such as insufficient inventory, product delisting, payment failure, payment timeout, and SKU expiration, avoiding the design of only ideal paths while ignoring boundary conditions in real-world business operations.

In the PRD design, this solution selects the product details page as a sample function, explaining the functional background, user goals, page fields, core interactions, business rules, error messages, acceptance criteria, and basic tracking points. Although the product details page is just a front-end page, it connects product information, SKUs, inventory, shopping cart, orders, and payment processes, thus belonging to the key functionality of the MVP phase.

In the technical understanding section, this solution analyzes API responses, pseudocode logic, and order status. Product managers don't necessarily need to write the complete code, but they do need to understand interface fields, price units, product status, SKU inventory relationships, order state machines, and basic business validation logic. Only by understanding these elements can they communicate more accurately with the development team and translate the technical logic into clear product rules and user prompts.

In the anomaly analysis section, this solution addresses the issue of "the product details page displays an inventory of 1, but an insufficient inventory message appears when submitting an order." This problem could stem from multiple sources, including caching, concurrent purchases, inconsistent SKU inventory, invalid items in the shopping cart, and the order submission interface re-verifying inventory. The core principle reflected in this section is that while front-end displays can serve as a user reference, real-time back-end verification must be the final basis for judgment.

In terms of data metrics, this solution includes core indicators such as product detail page views, add-to-cart rate, order submission conversion rate, payment success rate, GMV, order completion rate, order cancellation rate, and refund rate. It also uses conversion funnel analysis to pinpoint user churn points. These metrics help the team determine whether the system has truly achieved a seamless transaction loop after the MVP is launched and provide data support for subsequent iterations.

10.1 Core Product Judgment in the MVP Stage

The core product assessment in this plan is:

The first version of the MVP should not pursue full functionality, but should prioritize ensuring the stability and availability of the core transaction chain.

For a basic e-commerce system, the most important thing is that users can buy goods, operations staff can manage goods and orders, and the system can accurately handle inventory, payment, delivery, and order status. If these basic capabilities are unstable, even if coupons, membership systems, or referral systems are implemented, they cannot truly support business operations.

Therefore, the MVP phase should focus on the following four points:

1. **Is the transaction process running smoothly?** Can users successfully complete order placement, payment, and order inquiry from product browsing?
2. **Are the key states clear?:** Are product status, SKU inventory, order status, and payment status accurately synchronized?
3. **Can abnormal scenarios be handled?** Are there clear rules and prompts for issues such as insufficient inventory, product delisting, payment failure, and payment timeout?
4. **Are the data metrics traceable?** After the system goes live, can it use core metrics to determine at which stages users churn and whether the transaction loop is stable?

From a product manager's perspective, the value of an MVP lies not in "doing more," but in "getting the most critical issues right first." This solution places complex marketing, recommendation, membership, and advanced operational features in later versions to avoid making the first phase too broad, which could affect the efficiency of launching and validating core features.

10.2 Future Iteration Directions

After the MVP version is launched and the basic transaction process is validated, the system can continue to iterate in the following directions.

1. Improve the user purchase experience

Further optimizations can be made to features such as product search, category filtering, product reviews, favorites, coupons, product recommendations, and promotional activities. These capabilities will help users find products faster and improve purchase conversion rates. For example, product reviews can enhance user trust, coupons can stimulate orders, and recommendation systems can improve product discovery efficiency. However, these features

should be built after the basic transaction chain is stable, rather than investing too many resources in the first version too early.

2. Optimize inventory and order management capabilities

The MVP stage can initially support basic inventory maintenance and order processing, and can be further enhanced with capabilities such as inventory alerts, inventory flow tracking, batch inventory adjustments, order filtering, batch shipping, abnormal order marking, and logistics status synchronization.

These features can improve the efficiency of operations staff and reduce problems such as inaccurate inventory, order backlog, and shipping delays.

3. Improve after-sales and refund processes

Going forward, we can further improve our capabilities in areas such as refund reasons, refund review, return logistics, after-sales time limits, partial refunds, and handling failed refunds. The after-sales process directly impacts user satisfaction and platform trust; therefore, once the basic transaction process is stable, the clarity and efficiency of after-sales processing should be gradually improved.

4. Establish a more complete data analysis system

After completing the basic data tracking in the MVP phase, you can further build data dashboards and analysis systems, such as product conversion rate, SKU sales structure, reasons for payment failure, reasons for order cancellation, reasons for refunds, user repurchase rate, and channel conversion rate.

This data can help product and operations teams move from "being able to complete transactions" to "continuously optimizing conversion and growth".

10.3 Final Summary

Overall, this solution is based on the MVP model of an e-commerce system, and has been fully designed around user roles, functional priorities, core processes, simplified PRDs, API understanding, code logic, order status, exception analysis, and data metrics.

The focus of this solution is not on designing a complex, large-scale e-commerce platform all at once, but rather on ensuring the stable operation of the basic transaction loop. For the first version of the MVP, the most important thing is to enable users to purchase goods smoothly, to enable operations staff to manage goods and orders, and to enable the system to accurately handle inventory, payment, delivery, and order status.

Subsequent versions can further expand upon the MVP to include search recommendations, marketing tools, after-sales support, data analysis, and risk control capabilities. In this way, the system can first complete core validation from scratch, and then gradually move from basic usability to user experience optimization and business growth.

Therefore, the final idea of this solution can be summarized as follows:

1. First, ensure the transaction loop runs smoothly, then optimize the user experience;
2. First ensure accurate inventory and orders, then expand marketing capabilities;
3. First, validate the basic value through an MVP, and then drive subsequent iterations with data.